

25-1-25

Application

Application:-

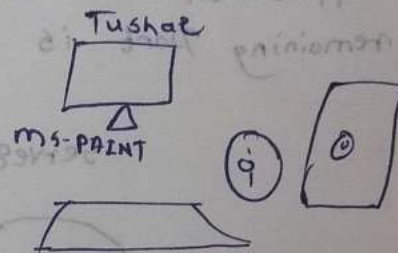
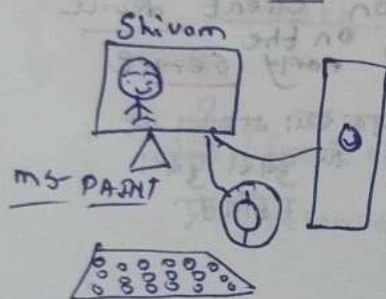
It is set of Program which is used to do perform a specific task

There are three types in application

- 1) STAND ALONE Application
- 2) web Application
- 3) client / server Application

1) STAND ALONE Application:-

- Any Application which can be used by only one user. This is known as STAND ALONE Application.



ex:- Notepad, Microsoft Office, VS Code, Music Player, Calculator.

Characteristics:-

- 1) Internet is not Required.
- 2) Installation is Required.
- 3) only one person can access at a time.
- 4) Database is not Required.

2) Web Application:-

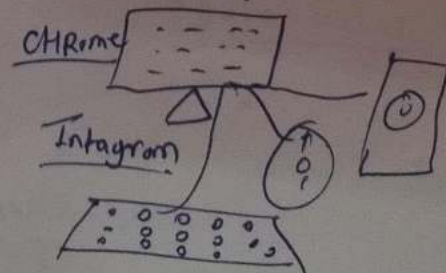
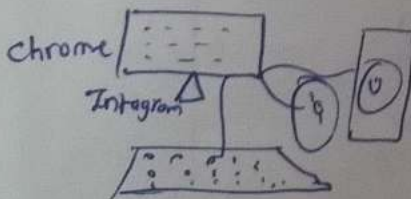
- Any Application which can be used through browser. It is known as web Application.

Krishna

Server

Instagram

Rushikesh



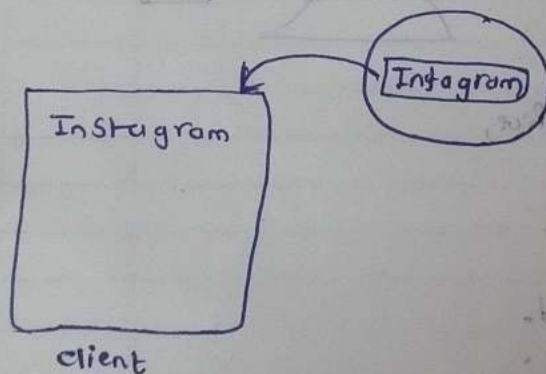
characteristics -

- 1) Internet is Required.
- 2) Installation is not Required.
- 3) multiple use access at a time
- 4) Database is Required.

★ Server :- It is nothing but super computer
 - It is a place where all the applications are installed.

★ client / Server application ★

A part of the application is installed on client device and the remaining part is installed on the server machine.

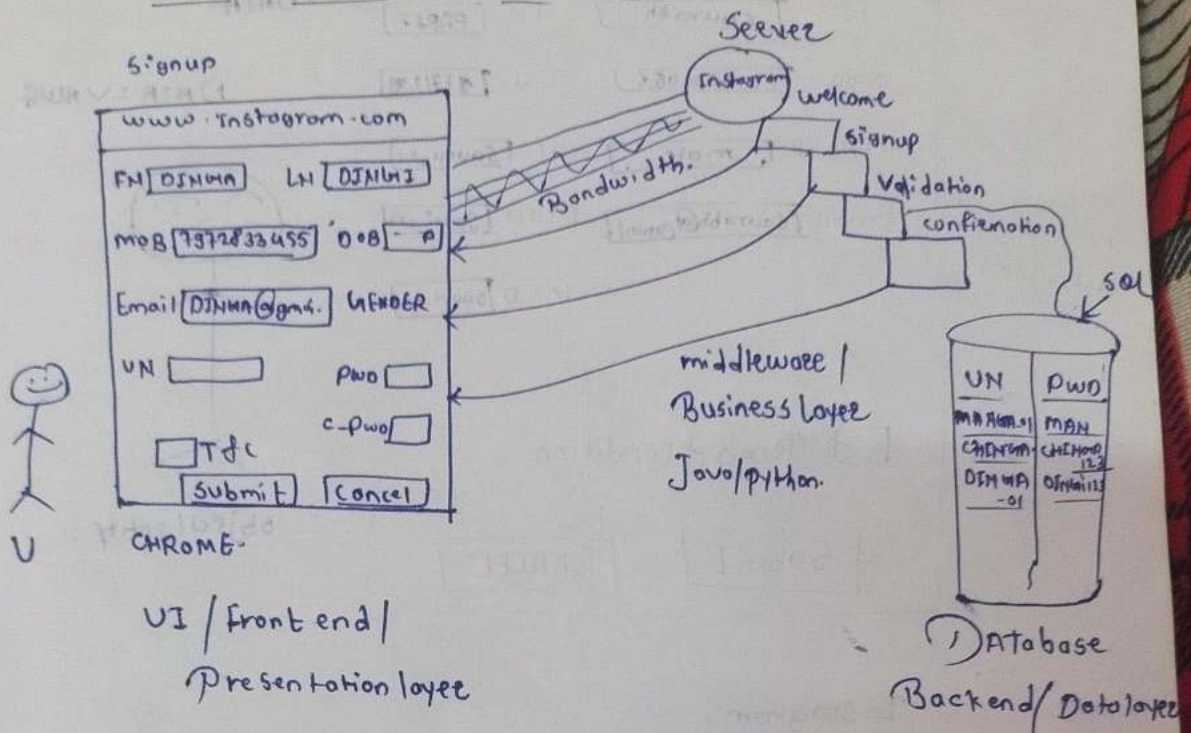


मक ह्या मशीन वर
 तर दुसरा दुसरा
 मशीन वर

characteristics

- 1) Internet is Required
- 2) installation is Required.
- 3) multiple use access at a time
- 4) Database is Required

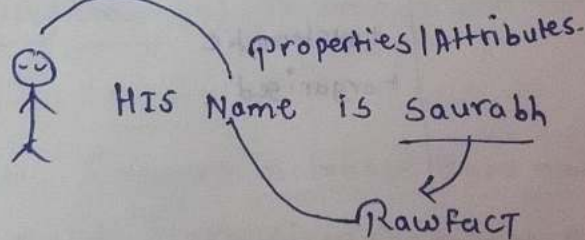
3-tier Architecture / n-tier Architecture



* Bandwidth is Nothing but Speed of Internet.

Data :- Data is a Raw fact which describes properties or attribute's of an object or entity.

Database object/entity



Name	Rushi
AGE	22
GENDER	male
DOB	11/12/2003
HEIGHT	6.0
WEIGHT	45
BLOOD GROUP	B+
MOB NO	7972833455
AADHAR	44113168859
PAN	LAWPK1520L
PL	
Email	

DATA = Value.

Data = value

Properties / Attributes

FN [Saurabh] LN [PARIF] DATA

Mob [797283456] DOB [11/1/2000] DATA = VALUE

GENDER [male] UN [Sauru_21]

Email [Saurabh@gmail] PWD [Sauru_123]

C-PWD [Sauru_123]

☒ Terms & condition

Submit CANCEL

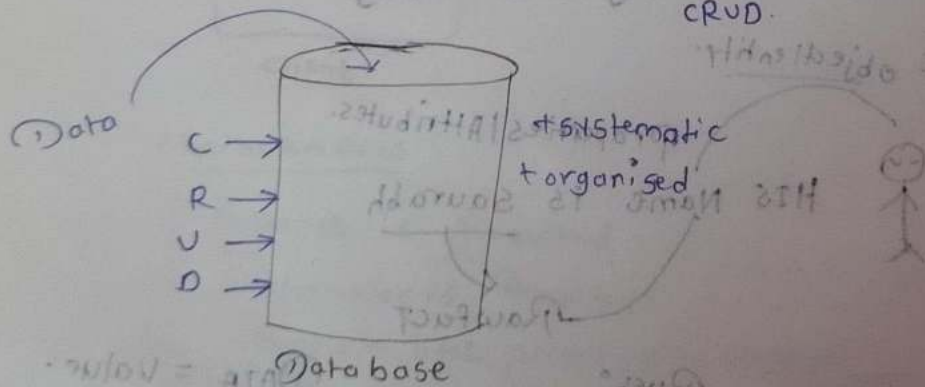
object entity

Instagram

DATABASE:-

It is a place where Data store's in a systematic and organised manner.

CRUD



These are the basic operations which we can perform on the Database and those are

CREATE / INSERT
READ / RETRIEVE
UPDATE / MODIFY
DELETE / DROP

This operation universally preferred as CRUD operations

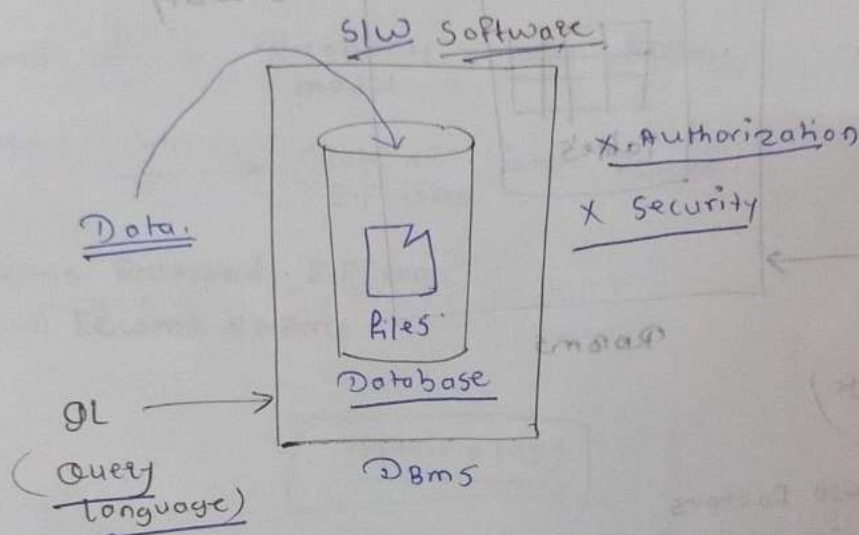
DBMS (Database management system)

- It is a software which is used to maintain and managed a Database.

- In DBMS Data Stores Files

करते Database को

आपको Data
Store करके
File में है.



It provides two factors -

- 1) Authorization
- 2) Security

OL में हमें अपने चयन में चयन करने
संभावना कर सकते

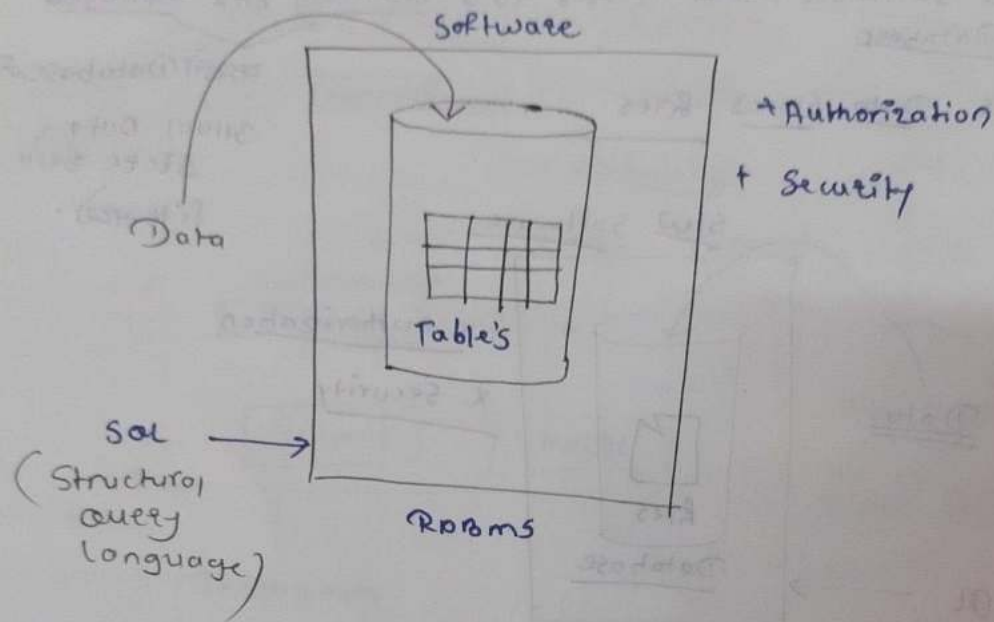
we use OL (Query language) To communicate with DBMS

Types of DBMS

1. NDBMS (Network Database Base management system)
2. HDBMS (Hierarchical Database Base management system)
3. RDBMS (Relational Database management system)
4. OODBMS (Object oriented Database management system)

ROBMS (Relational Database management system)

- It is a type of DBMS software.
- In ROBMS Data stores in the form of table's.



- It provide two Factors

- ① Authorization
- ② Security

we use SQL (Structured query language) to communicate with the ROBMS.

questions?

→ Interview question

① What is the use of SQL?

we use of ROBMS to communicate the ROBMS. with help of SQL.

② Difference between DBMS and ROBMS? → Interview question

DBMS

ROBMS

① DBMS stands for Database management system

① ROBMS stands for Relational Database management system.

② DBMS to communicate the query language use

② ROBMS to communicate the SQL is language is use.

③ Data is store in the form of files DBMS.

③ ROBMS data store in the form of the Tables.

④ Database is the software to maintain and manage data base.

④ ROBMS is the type of DBMS software.

Relational model

It is designed by a Data scientist called as E.F. Codd
(Edge Frank Codd)

- E.F. Codd
- In Relational model Data stores in the form of tables
- Any DBMS Follows Relational model it will become RDBMS

DBMS $\xrightarrow{\text{Follow}}$ Relational model $\rightarrow$ RDBMS

DBMS $\xrightarrow{\text{Follows}}$ Role of E.F. Codd $\rightarrow$ RDBMS

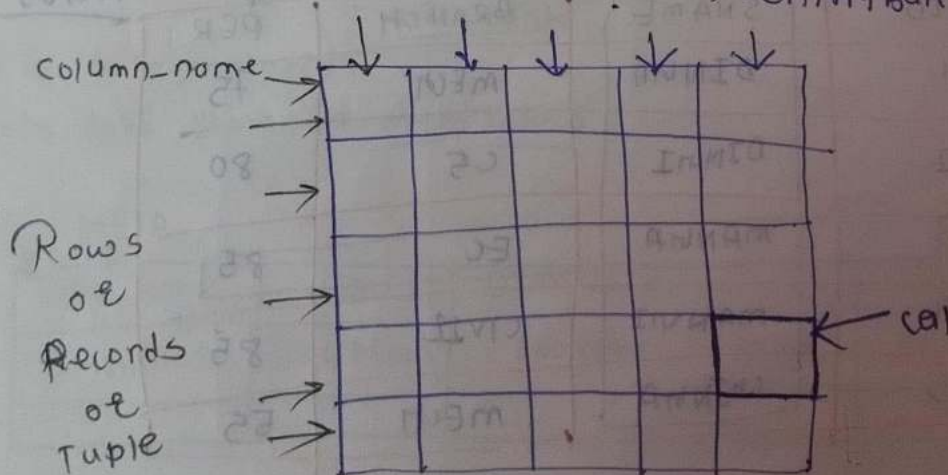
Any DBMS Followed E.F. Codd

- It will become RDBMS

Terminology

TABLE: The logical organization of data which consists Rows and columns it's known as Table
OR

The combination of Rows and Column is known as Table
column or Attribute / properties or field.



Columns

- It is also referred as properties, Attributes, fields.

- It represents of single property of all the the entity

Rows

आपकी Property
मथून सबकी show कराने

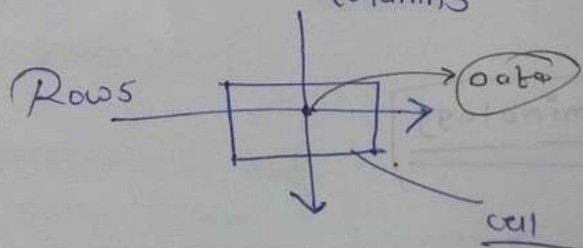
- It is also referred as Records or tuples

- It represent all the properties of a single entity

cell

- It is the smallest unit of the Table

- The interseccion between Rows or columns - It will generate cell.



It is nothing but Any
thing which has
existence.

* entity - It is nothing but Any things which has
existence.

Col-Name	STUDENT			
	SID	SNAME	BRANCH	PER
Rows or Record or Tuples	1	DINWA	MECH	75
	2	DINGI	CS	80
	3	MANGA	EC	85
	4	MANWI	CIVIL	85
	5	MAJWA	MECH	55

RULES OF EF CODD

- ① The data entered into the cell must be a single valued data.
To avoid data loss.

Student

SIO	SNAME	MOB-NO
1	ROHIT	123
2	SAKSHI	124
3	VINEET	125 (126)
4	KAREEN	127

(129)

STUDENT

SIO	SNAME	MOB-NO	Alt-MOBNO
1	ROHIT	123	-
2	SAKSHI	124	-
3	VINEET	125	(126)
4	KAREEN	127	-

129

- 2) In DBMS we store every thing in the form of Tables including meta data.

STUDENT

SIO	SNAME	Photo
1	SALLU	
2	AISH	
3	VIVEK	
4	KAT	

METADATA

Img-NAME: Img_002
TIME: 04:05pm
DIMENSION: 400x600
SIZE: 2MB
DEVICE: SAMSUNG
LOC: DANGE CHOWK

metadata:- ① Data about the data or

② Details about the data is known as meta data.

- It is autogenerated.
- meta data stores in meta tables.

META

Img-NAME	TIME	DIMENSION	SIZE	DEVICE	LOC
Img_002	04:05pm	400x600	2MB	SAMSUNG	DANGE CHOWK

- 3) According to e.f cood we can store the data in multiple Table if needed we can established connection between the Table by using Key attributes.

EMP

EMP	ENAME	ONo
1	Proshant	30
2	KunJon	20
3	AKANKSHA	30
4	SHARDUL	10
5	DEEP	30
6	KAREENA	20

DEPT

ONo	ONome	Loc
10	D1	PUNE
20	D2	G100
30	D3	BANGLORE

- ④ The data entered into the table can be verified in two Steps:-

- ① By assigning data types
- ② By assigning constraints.

note:- 1) Data Type are mandatory. Constraints are optional
2) SQL is not a case sensitive language

EMP

EMP	ENAME	SAL	HD	ONo
1	DEEP	750	18-Jan-98	30
KUNJAN	2	950	20	26-MAR-99
3	PARTH	18-Jan-99	1000	10
4	ARYAN	700	19-Jan-98	10
5	PRATIK	30	900	18-MAR-98
6	CHAITRA	950	18-Jun-99	20

* Data Type *

- It is used to describe what kind of type of data going to store in a particular memory location.

- There are 5 Types in Datatype

- 1) CHAR
- 2) VARCHAR / VARCHAR2
- 3) Number
- 4) Date
- 5) Large object.

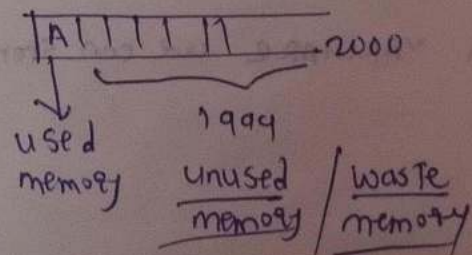
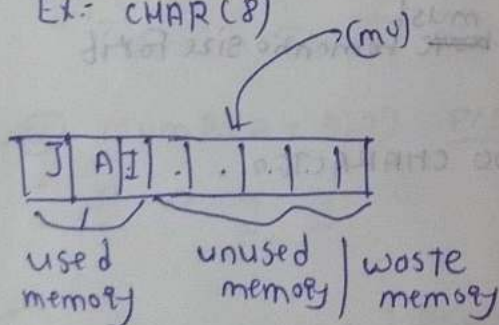
- ↓
- 1) CHARACTER large object.
 - 2) Binary large object

① CHAR :- It is used to store (Uppercase) 'AtoZ',
(Lowercase) 'atoz',
(number) '0to9',
Spe- Special character
Alphanumeric

Syntax:-

CHAR(SIZE)

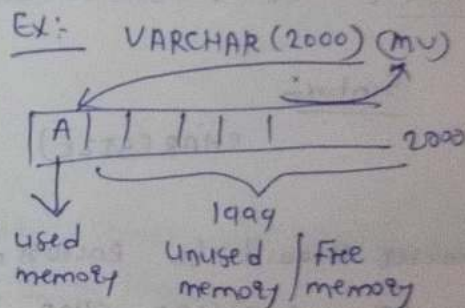
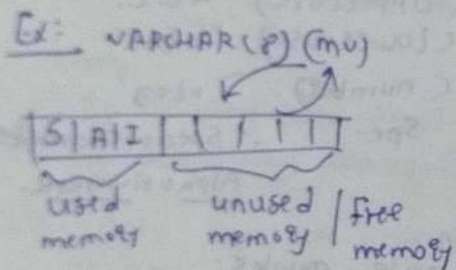
- character should be enclosed within single quotes.
- whenever we assign CHAR data type, we have to mention size of it.
- In CHAR data type upto 2000 CHARACTER.
- In CHAR the unused memory is also known as waste memory.
- Ex: CHAR(8)
- Ex: CHAR(2000)



It followed Fixed length memory allocation.

② VARCHAR / VARCHAR2 :

- It is used to store 'Atoz', 'atoz', 'atog', special character, 'alpha numeric'.
- Syntax: VARCHAR(size)
- Whenever we assign VARCHAR we have to mention size first.
- CHARACTER should be enclosed within single quotes.
- In VARCHAR we can store upto 2000 CHARACTER.
- It follows variable length memory allocation.
- In VARCHAR the unused memory is known as Free memory.



③ VARCHAR2 :

- It is an updated version of VARCHAR.

Syntax: VARCHAR2(size)

Whenever we assign to VARCHAR2 we ~~have~~ must mention size for it.

- In VARCHAR2 we can store upto 4000 CHARACTER.

3) DATE :-

- It is used to store date's.

- Syntax: DATE

Specified date format which is given by Oracle.

'00-mon-yy'

'00-mon-yyyv'

notes: Date should be enclosed with in single quotes

4) Number :-

- It is used to store numeric value.

Syntax: Number (precision, [SCALE])

Precision: - It is used to store integer value
- maximum size of Precision is 38.

Scale: - It is used to store decimal values.

- The maximum size of Scale is 127.

By default the scale value is 0.

Ex ① NUMBER (8,2) P75

+ 9 9 9 9 9 9 9
- 2 -1

Ex ② NUMBER (5,5) P=5

+ 9 9 9 9 9
- 5 -4 -3 -2 -1

Ex ③ NUMBER (5,0) P75

+ 9 9 9 9 9
- - - - -

Ex: ④ NUMBER (3,5) P<5

+ 0 0 9 9 9
- 5 -4 -3 -2 -1

0-00
↓ ↓
I D
↓ ↓
P S
↓ ↓
38 127

★ LARGE OBJECT :

1) CHARACTER LARGE OBJECT :

- It is used to store huge amount of character upto 4GB

Syntax: CLOB

2) BINARY LARGE OBJECT :

It is used to store image, audio, videos, files, records and etc. upto 4GB in the form of binary.

Syntax: BLOB

DATA TYPE	NUMBER(4)	VARCHAR(16)	NUMBER(3)	DATE	NUMBER(10)
COLUMN NAME	EID	ENAME	SAL	HIREDATE	MOBNO
	1011	ARIAN	15000	14-FEB-25	912345678
	(101)	NANDHANI	()	18-JAN-24	(812345)
	(10)	()	18000	()	()
	()	YUVRAJ	(-18000)	21-JUL-23	(912345678)

EMPLOYEE TABLE

CONSTRAINTS: constraints are the rules assigned to the particular column to validate data.

constraints are the rules assigned to the particular column to validate the data.

They are five types in constraints.

- ① UNIQUE
- ② NOT NULL
- ③ CHECK
- ④ PRIMARY KEY
- ⑤ FOREIGN KEY

① UNIQUE:

It is a constraints assigned to a column which should not accept duplicated or repeated values.

* It is constraints assigned to a column which should not be accept duplicated or repeated value.

② NOT NULL

It is o
- accept

③ CHECK:

It is o
IF
else n

DATA TYPE

COLUMN
NAME

UNIQUE

NOT NULL

CHECK

↓

cons

④ Primary

to id

★ character

✓ ① we

✓ ② It w

✓ ③ It

④ It

⑤ Pr

② NOT NULL:

It is a constraint assigned to a column which should not accept null value.

③ CHECK:

It is a constraint assigned to a column for extra validation. If the condition is satisfied, the value will be accepted else rejected.

DATA TYPES	NUMBER (4)	VARCHAR (15)	NUMBER (2)	DATE	NUMBER (9)
COLUMN NAME	EMPID	ENAME	SAL	HIRE DATE	MOB NO
UNIQUE	UNIQUE				UNIQUE
NOT NULL	NOT NULL	NOT NULL	NOT NULL	NOT NULL	NOT NULL
CHECK	(CHECK LENGTH(EMPID)=4)		CHECK (SAL > 0)		CHECK (LENGTH(MOBNO)=10)

Constraints

④ Primary Key:

It is a constraint assigned to a column which is used to identify the record's uniquely from the tables.

★ Characteristics:-

- ✓ ① We can have only one primary key in a table.
- ✓ ② It will not accept duplicated or repeated value.
- ✓ ③ It will not accept NULL.
- ④ It is a combination of unique and not NULL constraints.
- ⑤ Primary key is mandatory but highly recommend to provide.

Data type	NUMBER(4)	VARCHAR(15)	NUMBER(8)	DATE	NUMBER(4)
Column name	EID	ENAME	SAL	HIRE DATE	MOB No
UNIQUE					unique
NOT NULL	NOT NULL	NOT NULL	NOT NULL	NOT NULL	NOT NULL
CHECK	CHECK (LENGTH(EID)=4)		CHECK (SAL > 0)		CHECK (length > 10)
Primary key	Primary key				
Foreign key					
↓					
CONSTRAINTS					

Emp TABLE

⑤ FOREIGN KEY:-

It is used to establish connection between the tables.

Characteristics:

- ① we can have multiple Foreign key in a table. ✓
- ② It will accept duplicated or repeated values. ✓
- ③ It will accept NULL ✓
- ④ It is not a combination of unique & NOT NULL constraints. ✓
- ⑤ For an attribute to become a foreign key, it should be primary key in its own table.
- ⑥ Foreign key present in child table but actual belongs to parent table.

⑦ It is also known as Refer

Integrating constraints

Emp

EID

1

2

3

4

5

6

7

8

Primary key.

Q:- Different

① This is a

Column

Unique

② There

Primary

③ It has

duplicated

④ It is

⑤ It is

NOT

Emp	ENAME	PID	DNO
1	JETHAJI	101	20
2	BABITAJI	NULL	10
3	IYER	101	30
4	DIBBA	NULL	10
5	CHAMPK	102	30
6	MADHAVI	101	20
7	GOLI	NULL	10
8	HATHI	102	20

Foreign Key Foreign Key

Primary Key

CHILD TABLE

Product	Dept			
PID	PNAME	DNO	DNAME	LOC
101	P1	10	D1	C1
102	P2	20	D2	C2
		30	D3	C3

Primary Key

PARENT TABLE

Primary Key

PARENT TABLE

Q:- Difference between Primary key & foreign key. ? Interview question

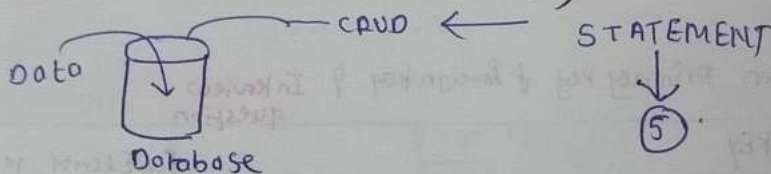
PRIMARY KEY	FOREIGN KEY
① This is a constraint assigned to a column which is used to uniquely identify records from the table.	① This is used to establish a connection between the tables.
② There can be only one Primary Key in the table.	② There can be multiple Foreign Key in a table.
③ It will not accept duplicated or repeated values.	③ It will accept duplicated repeated values.
④ It will not accept NULL.	④ It will accept NULL.
⑤ It is a combination of unique & NOT NULL constraints.	⑤ This is NOT a combination of unique & NOT NULL constraints.

Statements: It help us to perform "crud" operation on the Database.

Overview of SQL Statements:

There are Five statement in SQL

- ① DDL (Data Definition Language)
- ② DML (Data Manipulation Language)
- ③ TCL (Transaction control Language)
- ④ DCL (Data Control Language)
- ⑤ DQL (Data Query Language)



① DQL (Data Query Language) :- It is use to Retrieve the Data from the Database.

There are Four statement in DQL.

- 1) SELECT.
- 2) PROJECTION.
- 3) SELECTION.
- 4) JOINS.

1) SELECT: This statement is use to Retrieve the Data from the Tables and display it.

2) PROJECTION: It is used to Retrieve Data from the table by selecting only column.

- In Projection the data present under the column will be selected by default.

3) SELECTION: It is use to Retrieve the data from the table by selecting Row as well as column.

4) JOINS: It is use to Retrieve from the data from the multiple tables simultaneously.

PROJECTION:- This is used to Retrieve Data From the Table by selecting only column.

Syntax:-

SELECT * / [DISTINCT] column_Name / Expression [Aliases]
FROM TABLE_NAME;

ORDER OF EXECUTION

- 1 - FROM
- 2 - SELECT

STATEMENT

Q:- write a query to display STUDENT NAME.

② SELECT SNAME

① FROM STUDENT;

DATABASE

o/p of from clause.

SID	SNAME	BRANCH	PER.
1	Gopi	CS	55
2	RASHI	IT	80
3	KOKILA	CIVIL	75
4	JIGAR	MECH	85
5	AHEM	MECH	70
6	KINJAL	EC	80

STATEMENTS = CLAUSE

o/p of SELECT CLAUSE

SNAME
Gopi
RASHI
KOKILA
JIGAR
AHEM
KINJAL

Result Table.

Q:- write a query to display percentage of the student.

→
SELECT PER
FROM STUDENT;

SELECT PER
FROM STUDENT;

Q:- write a query to display SID and SNAME of the student.

→

SELECT SID, SNAME
FROM STUDENT;

Q:- write a query to display percentage and branch of the student.

→

SELECT PER, BRANCH
FROM STUDENT;

OR OF SELECT CLAUSE:

PER	BRANCH
55	CS
80	IT
75	CIVIL
85	MECH
70	MECH
80	EC

select clause
is responsible
for the Result
table.

Result table:

* SELECT CLAUSE is responsible for Result table.

Q:- write a query to display STUDENT_ID, BRANCH, student-name, percentage.
(WQTD)

→

SELECT SID, BRANCH, SNAME, PER
FROM STUDENT;

* All column select
in the student.

Q:- WQTD All the details of the student table.

SELECT *
FROM STUDENT;

Q: WRTD EMPNAME, SALARY, HIREDATE, and department number of Emp.

→

```
SELECT EMPNAME, SAL, HIREDATE, DEPTNO  
FROM Emp;
```

Q: WRTD EMPID, EMPNAME, SAL, along with ^{their} Disignation

```
SELECT EmpID, EMPNAME, SAL, JOB  
FROM Emp;
```

NOTE:-

- 1) From clause start execution.
- 2) For From clause we can pass on Table name as an argument (input)
- 3) The Job of From clause go to the Database and search for the table and put the table under execution.
- 4) Select clause execute After the execution of From clause.
- 5) For select clause we can pass * / Column-name / Expression
- 6) The Job of select clause it will goes table in which it is under execution and select the column which is mention and display it.
- 7) By this we can say select clause is Responsible for the Result table.
- 8) Semicolon (;) : It is nothing but end of the query or statement.
- 9) Asterisk (*) : It will select All column in table.

Q: WRTD All Tables Present in the user.

→

```
SELECT *
```

```
FROM TAB;
```

Q: WRTD All the details of the Dept Table. column number

```
SELECT *
```

```
FROM DEPT;
```

DEPTNO DNAME LOC.

Q: WRTD All the details of the Emp Tables

```
SELECT *
```

```
FROM Emp;
```

EMPID

EMPNAME

JOB

MGR

HIREDATE

SAL

COMM

DEPTNO

Q: WARETD EMPID, ENAME, Salary, and along with, annual salary of emp?

SELECT EmpID, ENAME, SAL, SAL*12
FROM Emp;

SELECT EmpNO, ENAME, SAL, SAL*12
from

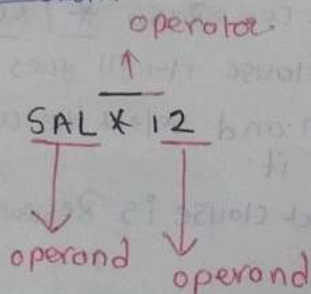
* Expression *

The combination of operand and operator is known as expression

A statement which gives result is known as Expression

OR

the combination of operand and operator's is known as Expression.



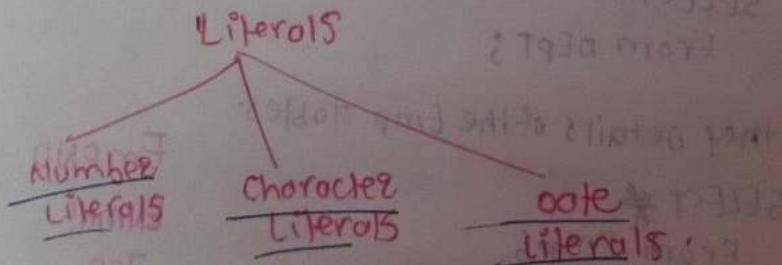
There are Two Types in expression

- ① operand
- ② operator (+, -, *, /)

1) operand:- operand consist Two types

- 1) column name
- 2) Literals (direct value)

- Literals consist Three types.



Date Literals:- Date format is which is given by oracle.

1) 'YY-MM-DD'

2) 'YY-MM-DD'

emp ?

Note: character literal and date literal enclose with in single quotes -

Q:- WAQTO EMPNAME, HIREDATE, Designation, salary, and half turn salary. Empname

→
 SELECT EMPNAME, HIREDATE, JOB, SAL, SAL*6
 FROM EMP;

Q:- WAQTO EMPID, EMPNAME, SAL, and along with 5% hike in the salary

SELECT EMPNO, EMPNAME, SAL, SAL*(SAL*0.05)
 FROM EMP;
 salary + salary * $\frac{5}{100}$

Q:- WAQTO EMPNAME, SAL, and 25% deduction in the salary

SELECT EMPNAME, SAL, SAL - (SAL * $\frac{25}{100}$)
 FROM EMP;

Q:- WAQTO EMPNAME, HIREDATE, SAL, and 50 rupees bonus in the annual salary

SELECT EMPNAME, HIREDATE, SAL, (SAL*12) + 50
 FROM EMP;

Q:- WAQTO All the details of emp along with annual salary.

SELECT *, SAL*12
 FROM EMP;

TableName.colname

↑
syntax

Syntax:

Tablename.colname

↑
wrong

↑
write

(X) ALIAS (X)

- It is an alternative name which is given to column or expression in the Result table.

- ① with or without AS keyword to assign ALIAS.
- ② ALIAS should be one word or it should be enclosed within double quotes. or we can make it as single word by using (_) underscore.

Homework

Q1) WAQTD ENAME, AND SAL AND ALSO SAL WITH 18% HIKE

→

```
SELECT ENAME, SAL, SAL + SAL *  $\frac{18}{100}$ 
```

```
FROM Emp;
```

Q2) WAQTD ENAME AND SAL AND ALSO SAL WITH 40% deduction

```
SELECT ENAME, SAL, SAL - SAL *  $\frac{40}{100}$ 
```

```
FROM Emp;
```

Q3) WAQTD ENAME AND SAL AND ALSO SAL WITH 7% HIKE

```
SELECT ENAME, SAL, SAL + SAL *  $\frac{7}{100}$ 
```

```
FROM Emp;
```

Q4) WAQTD ENAME AND Annual SAL AND ALSO with 34% HIKE

```
SELECT ENAME, SAL * 12, SAL + SAL *  $\frac{34}{100}$ 
```

```
FROM Emp;
```

Q5) WAQTD ENAME AND Annual SAL with Bonus of 5000

```
SELECT ENAME, SAL * 12 + 5000
```

```
FROM Emp;
```

Q6) WAQTD NAME, SAL, OF THE Employee along with their Annual Salary

→

```
SELECT NAME1 SAL * 12
```

```
FROM Emp;
```


Q:- WA@TO ENAME AND JOB For ALL The employee with their Half Term Salary

```
SELECT ENAME, JOB, SAL*6  
FROM Emp;
```

Q:- WA@TO ALL the details of the employee Along with their Annual Bonus of 2000.

```
SELECT Emp. *, SAL*12+2000  
FROM Emp;
```

Q:- WA@TO NAME SALARY, And SALARY with HIKE of 10%

```
SELECT ENAME, SAL+SAL* $\frac{10}{100}$   
FROM Emp;
```

Q:- WA@TO NAME And salary with deduction of 25%.

```
SELECT ENAME, SAL-SAL* $\frac{25}{100}$   
FROM Emp
```

Q:- WA@TO NAME AND salary with monthly HIKE of R550.

→

```
SELECT ENAME, SAL+500*12  
FROM Emp;
```

Q:- WA@TO NAME AND Annual Salary with deduction of 10%.

→

```
SELECT ENAME, SAL*12-SAL*12* $\frac{10}{100}$   
FROM Emp;
```

Q:- WA@TO details of All the employee Along with Annual Salary.

→

```
SELECT Emp. *, SAL*12  
FROM Emp;
```

Q:- WA@TO NAME AND Designation Along with 100 RS. penalty In Salary.

→

```
SELECT ENAME, JOB, SAL-100  
FROM Emp;
```


Q:- WAO TO DIFFERENT SALARY.

SELECT DISTINCT SAL
FROM Emp;

* DISTINCT *

- DISTINCT is a clause which is used to remove duplicated or Repeated values from the Result Table.
- we can pass DISTINCT clause in the SELECT clause as a First (1st) argument.
- For distinct clause we can pass column name or expression as an argument.
- For distinct clause we can pass multiple argument but it will remove duplicated in the combination.

(I)

Q:- WAO TO DIFFERENT SALARY.

SELECT DISTINCT SAL

FROM Emp;

DATABASE

o/p of From

clause

(II) Emp

Emp	ENO	ENAME	SAL	DD
1	1	MAHENDRA	2500	30
2	2	KATAPPA	9000	20
3	3	SHIVGAMI	2500	30
4	4	BHALLAL	6000	10
5	5	DEVSENA	3000	20
6	6	KALKEY	5000	30

o/p of SELECT clause.

III

SAL
2500
4000
2500
5000
3000
5000

Result Table

o/p of DISTINCT clause

IV

SAL
2500
4000
5000
3000

Q:- WAATO different salary & DNO

→

II

② SELECT DISTINCT SAL, DNO
① FROM Emp;

III

o/p of SELECT clause

SAL	DNO
2500	30
4000	20
2500	30
5000	10
3000	20
5000	30

Result Table

SAL DNO both same then Remove
only one is match doesn't remove.

o/p of DISTINCT clause

SAL	DNO
2500	30
4000	20
5000	10
3000	20
5000	30

Note: After Result then that time's only DISTINCT query executed.
 But when you can run query that times Unique constraints executed.

Q1) WANTED The different designations that are present in Emp Tables.

→

```
SELECT DISTINCT JOB
FROM emp;
```

Q2) WANTED different DEPTNO as well as salaries that are present in the emp tables.

→

```
SELECT DISTINCT deptno, SAL
FROM emp;
```

Q3) WANTED All the details of the employee along with his Annual Salary.

→

```
SELECT emp.*, SAL*12 AS Annual_Salary
FROM emp;
```

Q4) WANTED All the details of employee along with 10% hike in the salary.

→

```
SELECT emp.*, SAL + SAL * 10 / 100 AS HIKE
FROM emp;
```

Q5) WANTED NAME of the employees who works in dept 20.

→

```
SELECT ENAME
```

```
FROM emp
```

```
WHERE DNO=20;
```

ENAME	DNO
SCOTT	10
JOHN	10
ADAMS	20
JAMES	20
SMITH	20
WATSON	20
MILLER	20

"SELECTION"

- It is use to Reterive the data From the that table by Selecting Row's as well as column.

* WHERE CLAUSE *

- It is use to Filter the conditions
- It execute's Row by Row
- It execute's After execution of FROM CLAUSE.
- In where clause we can write Filter condition which Returns boolean Values (TRUE or FALSE)
- In where clause we can Pass multiple condition by using logical operators

Q:- WA@TO NAME of the employees who works in dept 20.

SELECT ENAME.

Syntax:

SELECT * / [DISTING] column-NAME / Expression [ALIAS]
FROM TABLE-name.
WHERE <Filter-condition>;

order of Execution.

- 1- FROM
- 2- WHERE - - Row by Row -
- 3- SELECT

Q:- WA@TO NAME of the employees who work in dept 20.

③ SELECT ENAME

① FROM EMP

② WHERE DNO = 20;

Filter-condition

Emp

DATABASE ON PD

OIP of from clause.

EID	ENAME	ON
1	STARK	20
2	STEVE	30
3	BROCE	20
4	WAND	10
5	LOKI	10
6	GAMORA	30

OIP of WHERE CLAUSE

EID	ENAME	ON
1	STARK	20 = 20
2	STEVE	30 = 20
3	BROCE	20 = 20
4	WAND	10 = 20
5	LOKI	10 = 20
6	GAMORA	30 = 20

OIP of SELECT CLAUSE

ENAME
STARK
BROCE

Q1) WQTO The Annual salary of the employee whose Name is Smith.
 →
 SELECT SAL * 12 AS ANNUAL-SAL.
 FROM Emp
 where ENAME = 'smith';

Q2) WQTO NAME of the employees working as clerk.
 →
 SELECT ENAME
 FROM Emp
 where JOB = 'clerk';

Q3) WQTO SALARY of the employee who are working as Salesman.
 →
 SELECT SAL
 FROM Emp
 where JOB = 'salesman';

Q4) WQTO Details of All The employee who earns more than 2000
 SELECT *
 FROM Emp
 where SAL > 2000;

Q5) WAQTD details of the Employees whose NAME is JONES.

```
SELECT *  
FROM Emp  
WHERE ENAME = 'JONES';
```

Q6) WAQTD details of the employees who HIRED AFTER 1981.

```
SELECT *  
FROM Emp  
WHERE HIREDATE > '1981-01-01';
```

```
SELECT *  
FROM Emp  
WHERE HIREDATE > '31-DEC-81';  
WHERE HIREDATE > '01-JAN-82';
```

Q7) WAQTD NAME AND SALARY along with Annual Salary IF Annual SAL IS more IS more than 12000.

```
SELECT ENAME, SAL, SAL * 12 AS Annual-SAL  
FROM Emp  
WHERE SAL * 12 > 12000;
```

Q8) WAQTD Empno of the employees who are working in department NO 30

```
SELECT Empno  
FROM Emp  
WHERE DEPTNO = 30;
```

Q9) WAQTD details of the employees HIRED Before 1981

```
SELECT *  
FROM Emp  
WHERE HIREDATE < '01-JAN-81';  
WHERE HIREDATE < '01-JAN-1981';
```

Q10) WAQTD name and Salary given to the employees IF they HIRED AFTER 1982

```
SELECT ENAME, SAL  
FROM Emp  
WHERE HIREDATE > '31-DEC-82';
```

Q11) WAQTD details of the employees IF the emp HIRED AFTER 2ND month of 1981

```
SELECT *  
FROM Emp  
WHERE HIREDATE > '28-FEB-1981';  
WHERE HIREDATE > '01-MAR-1981';
```


OPERATORS

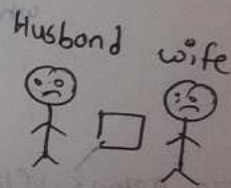
- ✓ ① Arithmetic operators (+, -, *, /)
- ✓ ② Concatenation operator (||)
- ✓ ③ Logical operators (AND, OR, NOT)
- ✓ ④ Comparison operators (=, !=, <, >, <=, >=)
- ✓ ⑤ Relational operators (<, >, <=, >=)
- ✓ ⑥ Special operators (IN, NOT IN, BETWEEN, NOT BETWEEN, LIKE, NOT LIKE, IS, IS NOT)
- ⑦ Subquery operators (ALL, ANY, EXISTS, NOT EXISTS)

① Logical operators

2) AND

Truth Table For AND

condition 1	condition 2	Result
T	T	T
T	F	F
F	T	F
F	F	F



note: In case of AND operator we get the output only if all the conditions are satisfied or True.

Q: Write a query to find the name, job, and salary of the employees who are working as salesmen and earn a salary greater than 1500.

③ ← Select ename, job, sal

① ← from emp

② ← where job = 'SALESMAN'
AND sal > 1500;

o/p: ENAME
ALLEN

JOB SAL
SALESMAN 1600

2) OR

condition
T
T
F
F

Note: In case of the condition

Q: Write a query to find the name of the salesmen who are

→

Q: Write a query to find the name of the salesmen who are

→

3) NOT

Note: N

Q: Write a query to find the name of the salesmen who are

→

2) OR

Truth table for OR

condition 1	condition 2	Result
T	T	T
T	F	T
F	T	T
F	F	F

Note: In case of or operators we get they output. If any one of the condition is satisfied or True.

Q: WA@TD Name, And Job of the employee who are working as Salesman or managers.

→

③ - Select Ename, JOB

① - From Emp

② - where JOB = 'SALESMAN' OR JOB = 'MANAGER';

Q: WA@TD details of they employee who are Salesman or they should working in DeptNo 30

→

Select *

From emp

where JOB = 'salesman' or DEPTNO = 30;

3) NOT

Truth table for NOT.

Result	Final Result
T	F
F	T

Note: NOT operator is used to Reverse to Results

Q: WA@TD Name and JOB of they employees except for managers

→

① Select Ename, JOB

① From emp

② where ~~NOT~~ JOB = 'MANAGERS';

② OR

F → T
T → F

② Select ename, job
from emp
where job != 'MANAGER';

OR

③ select ename, job
from emp
where job <> 'MANAGER';

Q: WAQTD name, Job and DeptNo of employee who are working
→ in dept No 20, 30

select ename, job, DEPTNO
from emp
where DEPTNO = 20 OR DEPTNO = 30;

Q: WAQTD They employee's who are Hired on '28-Sep-81' and
'3-Dec-81'

→ select *
from emp
where HIREDATE = '28-Sep-81' OR HIREDATE = '3-Dec-81';

Q: WAQTD name, and Job of they employee's who are working
as a clerk in deptno = 10

→ select Ename, Job
from emp
where Job = 'CLERK' AND DEPTNO = 10;

Q: WAQTD name, Job, and annual salary of they employee's
who annual salary is greater than 130000 thousand?

Select ename, job, sal * 12 as annual_sal
From emp
where sal * 12 > 15000;

Q: WAQTB The details of the employee who are working in deptno
10 or 20 work as manager or salesman and earn
salary lesser than 2000

Select *
From emp
where DeptNo = 10 AND DeptNo = 20
OR JOB = 'MANAGER' AND JOB = 'SALESMAN'
OR SAL < 2000;

Q: WAQTB details of the employee who are hired in year 81

Select *
From emp
where HIREDATE > '01-JAN-1981'
AND HIREDATE < '01-JAN-1982'; OR <= '31-DEC-81';

note AND:-

- It is used whenever we have to satisfied both the condition or
It Return T if it's satisfied both the conditions.

1) It can be return in between the condition

OR

- It is used whenever we have to satisfied Any one of the condition
or

It Return T if it satisfied Any one of the condition.

- It can be Return in between the condition.

NOT

It is used whenever we want to Inverse or Reverse the conditions

Q: WAO TO details of emp if the employee works as manager
if deptNO = 20 OR 30

→

```
SELECT *  
FROM emp  
WHERE JOB = 'MANAGER' AND  
(DEPTNO = 20 OR DEPTNO = 30); *only 53;
```

Q: WAO TO All details of the employee if the employee HIRE after
81 but before 86

```
SELECT *  
FROM emp  
WHERE HIREDATE > '01-JAN-81' AND  
HIREDATE < '01-JAN-86';  
OR  
HIREDATE > '31-DEC-81' AND  
< '01-JAN-86'
```

Q: WAO TO All the details of the employee except the employee
who work in deptNO = 20

```
SELECT *  
FROM emp  
WHERE NOT DEPTNO = 20;
```

* CONCATINATION OPERATORS *

- It is use to join the given strings

Symbol = ||

Syntax: string 1 || string 2

Example:

```
select 'MR.' || ENAME  
from Emp;
```

Q:- WATD Details of the employees If the employee's HIRE After 80 but before 87 and works as MANAGER, SALESMAN, clerk, AND the employee earns more than 1250 but less than 5000 AND the employee works in Dept NO 20, or 30 except the employee who works as ANALYST.

Select *

From emp

where HIREDATE > '31-DEC-80' AND HIREDATE < '01-JAN-87'

AND (JOB = 'MANAGER' ^{OR} JOB = 'SALESMAN' ^{OR} JOB = 'CLERK') AND

(SAL > 1250 AND SAL < 5000) AND

(DEPTNO = 20 OR DEPTNO = 30) NOT
JOB = 'ANALYST';

only
parenthesis
or operator

AND NOT

* Special operators *

① IN: It is the special operators which except multiple value values at R.H.S

- It Behaviour similar to equal operator
- It Returns TRUE if it satisfied Any one values of RHS.

Syntax:

column_name / Expression IN (v₁, v₂, v₃, v₄ ... v_n)

ENO IN (20, 30)

Q:- Write details of employee if the employee works as manager or salesman or Analyst or clerk and the employee should work deptno 20.

Select *

From emp

where JOB = 'Manager' OR JOB = 'SALESMAN' OR
JOB = 'Analyst'

AND DEPTNO = 20;

Select *

From emp

where JOB IN ('MANAGER', 'SALESMAN', 'ANALYST') AND
DEPTNO = 20;

@ NOT IN

It is similar to IN operator instead of selecting it will reject the values.

Syntax:-

column_name / Expression NOT IN (v1, v2, v3, ..., vn).

Q:- WAOB details of the employee except the emp who work's in DeptNo 20, 30

Select *
From emp

where DeptNo NOT IN (20, 30);

③ BETWEEN

- It is use whenever we have range of values.
- It include the given range.
- we can not interchange the range.

Syntax:-

column_name / Expression BETWEEN starting_value AND Ending_value

ex:-

SAL BETWEEN $\geq$ 1000 AND $\leq$ 3000

Q:- WAOB EMPID, EMPNAME, HIREDATE of the employee's If the employee earn's in the range of 1250 to 3000

Select EMPID, EMPNAME, HIREDATE
From emp

where HIREDATE BETWEEN 1250 AND 3000;
SAL

Q:- WAQTO details of the employee If the employee's HIRE in the year 1981

Select *

From emp

where HIREDATE BETWEEN '01-JAN-81' AND '31-DEC-81';

Q:- WAQTO EMPID, EMPNAME, HIREDATE, DEPARTMENT NO of the employee's If the employee's earn more than 1250 But less than 3000 and HIRE AFTER 80 but before 84.

Select * EMPID, EMPNAME, HIREDATE, DEPTNO

From emp

where SAL BETWEEN 1250 AND 2999 AND HIREDATE BETWEEN '01-JAN-81' AND '31-DEC-83';

* NOT BETWEEN *

- It is similar to Between operator. Instead of selecting it will reject the range (reject given range)

Syntax: column-name / Expression NOT BETWEEN starting_value AND Ending_value.

Q:- WAQTO details of the employee except hired in the '81' Accept the employee Hired

Select *

From emp

where HIREDATE NOT BETWEEN '01-JAN-81' AND '31-DEC-81';

(*) IS

- used to
- It is a special operator it is compare with NULL. ^{But empty not considered}

Syntax:-

column-name / Expression IS NULL

Q:- WRTD details of the employee whose commission is NULL.

```
select *
from emp
where comm IS NULL;
```

* ISNOT

- It is similar to IS operator but it will compare with NOT NULL value. ^{અથવા પૂર્ણ મૂલ્ય}

Syntax:-

Column-name / Expression IS NOT NULL.

Q:- WRTD All the details of the employee if the employee commission is not empty.

Full Value

```
select *
from emp
where comm IS NOT NULL;
```

Q:- WRTD All the details of the employee if the employee's earning more than 950 but less than 5000 and the employee's work in deptno 20 or 30 and the employee should HIRE after 80 but before 88 except the employee who HIRE in the 82 years and the employee should work as Analyst, manager, sales man, clerk and except the employee who is earning exactly 3000 and the employee should earn some comm.

```
select *
from emp
where SAL BETWEEN 951 AND 4999 AND
DEPTNO IN (20, 30) AND
HIREDATE BETWEEN '01-JAN-81' AND '31-DEC-81'
AND
```


HIREDATE NOT BETWEEN '01-JAN-82' AND
'31-DEC-82' AND

JOB IN ('Analyst', 'manager', 'Salesman', 'clerk')
AND

SAL ~~NOT BETWEEN~~ 3000 AND
comm IS NOT NULL;

* Like *

- It is used to perform Pattern matching

Syntax:-

Column_name/Exp LIKE 'Pattern_to_matching'

- To Achieve pattern matching there are two special characters
such as

Percentile & underscore

%. & -

① Percentile % (C%)

- It can accept n number of characters, Any number
of time or no character

② Underscore & (-) %

- It can accept any character but only one character

★ Examples :-

- 1) start with character A $\rightarrow$ 'A%'
- 2) end with character A $\rightarrow$ '%A'
- 3) Has character A $\rightarrow$ '%A%'
- 4) Has character A present twice $\rightarrow$ '%A%A%'
- 5) Has Two consecutive A $\rightarrow$ '%AA%'
- 6) start with A and end with k $\rightarrow$ 'A%k'
- 7) Second character is A $\rightarrow$ '_A%'
- 8) Third character is E $\rightarrow$ '___E%'
- 9) Third last character is E $\rightarrow$ '%E___'
- 10) start with k and second last character is N $\rightarrow$ 'k%N_'
- 11) Has exactly 5 characters '_____'
- 12) Second last character is E $\rightarrow$ '%E_'

⊗ Not Like ⊗

NOT LIKE operator is similar to like operator instead of selecting it will reject's the Pattern.

Syntax:-

(Column_name/Expression NOT LIKE 'Pattern-To-match';

HW

Q:- WA@TO details of the employees whose name starts with character A.

→

```
select *  
from emp  
where ENAME LIKE 'A%';
```

Q:- WA@TO NAME and salary given to the employees who are earning 3 digit salary.

Ans the
option
is
→
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U
V
W
X
Y
Z

```
select ENAME, SAL  
from emp
```

```
where SAL LIKE '____';
```

Q:- WA@TO details of the employee working in dept no 20 and ending with character S in their NAME.

```
select *
```

```
from emp
```

```
where deptno = 20 AND
```

```
ENAME LIKE '%S';
```

Q:- WA@TO details of the employee having character E as last second character in their JOB.

```
select *
```

```
from emp
```

```
where JOB LIKE '%E_';
```

Q:- WA@TO name of the employee having character A as the 3rd character in the NAME

```
select ENAME
```

```
from emp
```

```
where ENAME LIKE '___A%';
```


Q:- W@TO ENAME, AND HI@DATE of the employees who ^{anyone} ^{year} HI@ED during the year 82.

→
 select ENAME, HI@DATE
 From emp
 where HI@DATE BETWEEN '01-JAN-82' AND '31-DEC-82';

OR (LIKE 'A82')
 LIKE '1.82';
 01-JAN-82
 31-DEC-82

Q:- W@TO ENAME AND HI@DATE of the employee who were HI@ED in the month of FEB.

select ENAME, HI@DATE
 From emp
 where HI@DATE LIKE '1.1-FEB-82';

1.1-FEB-82
 1.1-FEB-82

Q:- W@TO ENAME, HI@DATE, AND SAL of the employees who are earning AS@l@ry of 3 digit And were HI@ED IN the month of DECEMBER

select ENAME, HI@DATE, SAL
 From emp
 where SAL LIKE '____' AND
 HI@DATE LIKE '1.1-DEC-82';

1.1-DEC-82
 1.1-DEC-82

Q:- W@TO All the details of the employees who work AS@ salesman And working in dept NO 30 and earn more than 1500

select *
 From emp
 where JOB = 'salesman' AND
 DEPTNO = 30 AND
 SAL > 1500;

Q:- W@TO details of all the employees whose name starts with 'A' OR 'S';

select *
 From emp
 where ENAME LIKE 'A%' OR ENAME LIKE 'S%';

Q:- WAOTD details of All the employees whose name Doesnot start with 'S'

```
Select *  
From emp  
where ENAME NOT LIKE 'S%';
```

Q:- WAOTD details of ALL the employees whose comission is null and working as a clerk.

```
Select *  
From emp  
where comm IS NULL AND  
JOB = 'CLERK';
```

Q:- WAOTD ENAME, SALARY, AND Annual Salary of the employees except those who are working in dept 30.

```
DEPTNO  
Select ENAME, SAL, SAL*12, as Annual-SAL  
From emp  
where DEPTNO != 30;
```

Q:- WAOTD All employees who HIRED After 81.

```
Select emp ENAME  
From emp  
where HIREDATE > '01-JAN-82' ; OR HIREDATE > '31-dec-81'
```

Q:- WAOTD NAME of the employees whose name start with vowels.

```
Select ENAME  
From emp  
where ENAME LIKE 'A%'  
OR ENAME LIKE 'E%'  
OR ENAME LIKE 'I%'  
OR ENAME LIKE 'O%'  
OR ENAME LIKE 'U%';
```

Q:- WAOTD All the details of the employees who HIRED AFTER 81
and Before 83 .

LIKE '%82';

```
Select *  
From emp  
where HIREDATE BETWEEN '01-JAN-82' AND '31-DEC-82';
```

Q:- WAOTD details of the employees whose JOB HAS STRING 'MAN'

```
Select *  
From emp  
where JOB LIKE '%MAN%';
```

Q:- WAOTD ENAME, Designation, and salary of the employee's whose name
Doesnot start with Vowel.

reject means
to write the
AND operators

```
Select ENAME, JOB, SAL  
From emp  
where ENAME NOT LIKE 'A%'  
AND ENAME NOT LIKE 'E%'  
AND ENAME NOT LIKE 'I%'  
AND ENAME NOT LIKE 'O%'  
AND ENAME NOT LIKE 'U%';
```

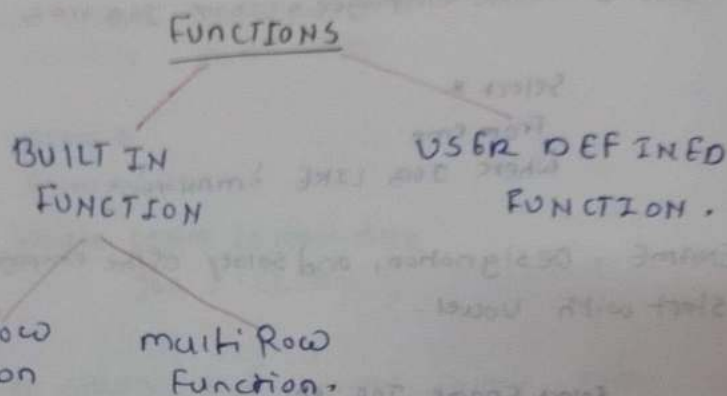
Q:- WAOTD ENAME, Designation Empno and salary of the employee
If the employee HIRED between 80 to 87 AND earns In The
range of 1000 to 3000 .

```
Select ENAME, JOB, EMPNO, SAL  
From emp  
where HIREDATE BETWEEN '01-JAN-80' AND '31-DEC-87' AND  
SAL BETWEEN 1000 AND 3000;
```


FUNCTIONS

x It is a set of instruction which is used to perform a specific operation.

- Function consists two type

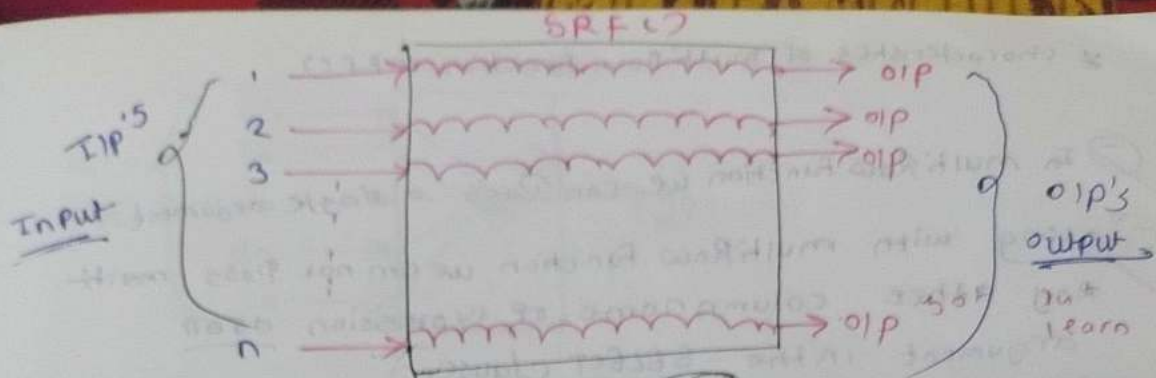


Built in Function consists two type

- ① Single Row Function
- ② Multi Row Function

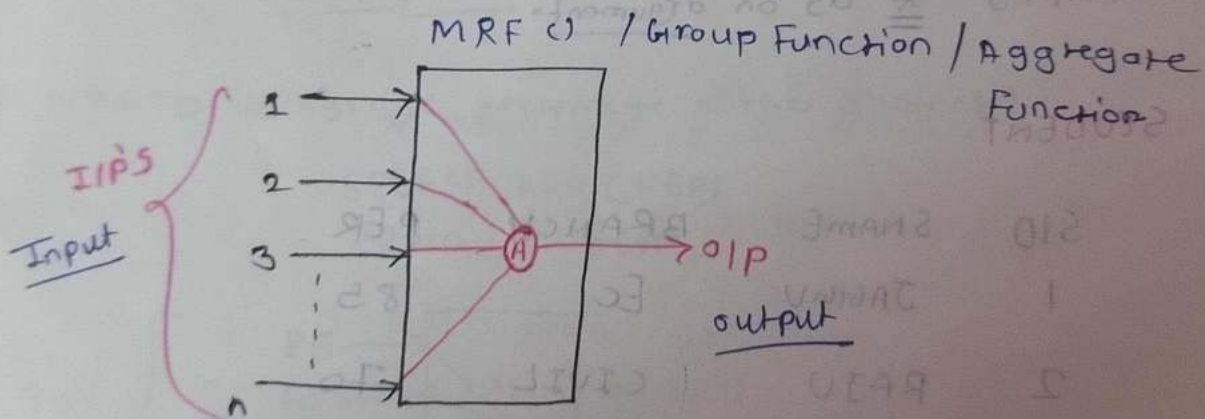
① Single Row Function :-

- It executes Row by Row
- It takes one Input and iterates and generate one output and goes to the next Input and goes on.
- If we pass 'n' number Input to single Row function it return 'n' number of outputs



Multi Row Function :

- It is also known as group function or aggregate function
- It executes group by group
- It takes all the input at once and aggregates it (combine) and generates a single output
- If we pass n number input to a multi row function it will return a single output



* LIST of MRF *

- 1) max ()
- 2) min ()
- 3) Avg ()
- 4) sum ()
- 5) count ()

* Characteristics of Multi Row Function MRF()

- ① In Multi Row Function we can Pass a single argument.
- ② Along with Multi Row Function we can not Pass maths Any other column name or expression as an argument in the SELECT clause.
- ③ Multi Row Function Ignores NULL.
- ④ In where clause we cannot pass Multi Row Function.
- ⑤ For sum(), avg(), we can Pass only number's as argument.
- ⑥ Count is the only Multi Row function which excepts * as an argument.

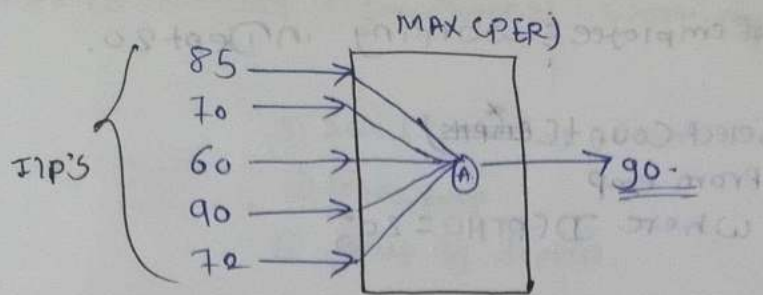
STUDENT

SID	SNAME	BRANCH	PER
1	JAGGU	EC	85
2	RAJU	CIVIL	70
3	BHEEM	IT	60
4	TUNTUN	MECH	90
5	KALIA	CIVIL	72

Q:- WAP TO maximum Percentage of the Student?

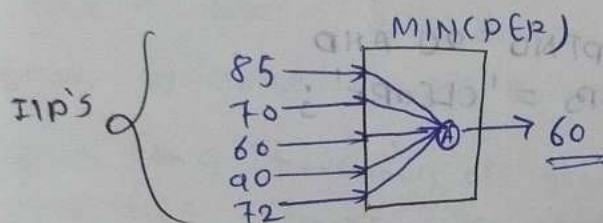
```
SELECT MAX(PER)
```

```
FROM STUDENT;
```



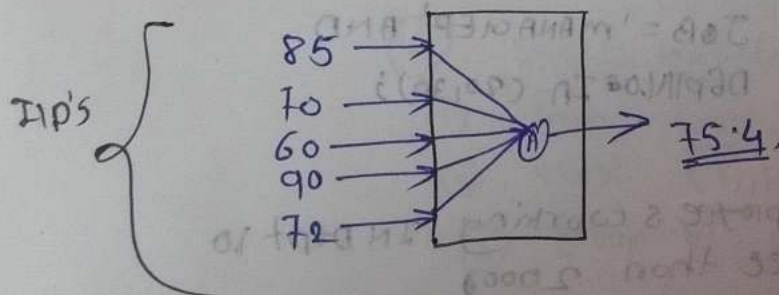
Q:- WAO To Minimum percentage of the student?

SELECT MIN(CPER)
FROM STUDENT;



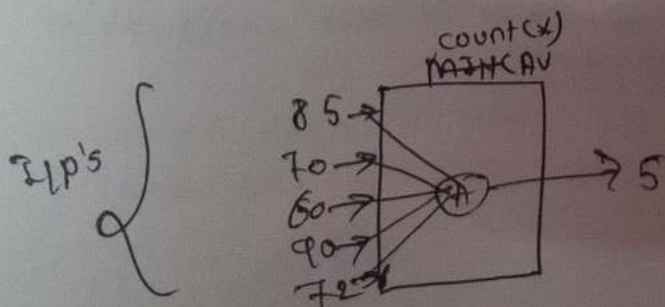
Q:- WAO To Average percentage of the student?

SELECT AVG(CPER)
FROM STUDENT;



Q:- WAO To number of Student present in the student Table?

SELECT COUNT(~~STUDENT~~^{IP})
FROM STUDENT; or select count(*)
FROM STUDENT;



Q:- WA@TD Number of employees working in Dept 20.

→

```
Select Count(*)
From emp
Where DEPTNO = 20;
```

Q:- WA@TD maximum Salary given To An Employee working In Dept 20 As a clerk.

```
Select max(SAL)
From emp
Where DEPTNO = 20 AND
      JOB = 'CLERK';
```

Q:- WA@TD total Salary given to the employees working As manager In Dept 20 OR 30.

```
Select SUM(SAL)
From emp
Where JOB = 'MANAGER' AND
      DEPTNO IN (20, 30);
```

Q:- WA@TD number of employees working IN Dept 10 with A SALARY greater than 2000.

```
Select Count(*)
From emp
Where DEPTNO = 10 AND
      SAL > 2000;
```

Q:- WA@TD Number

→

① group By
✓ kind of

② group by

③ after the
only has

④ Any clo
execute's

⑤ It execu

note:-

* The arg

It c

iti

exp

we

Q: What is the Number of employees working in Each Dept?

- ③ Select count (*) deptno
- ① from emp
- ② group by deptno;

* GROUP BY CLAUSE *

- ① group by clause is used to group or cluster similar kind of data.
- ② group by clause executes Row by Row and creates group's
- ③ after the execution of group by clause, the compiler only has group's with itself.
- ④ Any clause that gets executed after group by clause, executes group by group
- ⑤ It executes after execution of from clause.

Syntax:-

- ④ Select MRF / Group By column / Expression
- ① from table-name
- ② where <Filter condition>
- ③ group by column / Expression;

note:-

* The argument which we pass in the group by clause.

It can be passed in the select clause so it is known as group by column or group by expression. group by value group by clause must be in the select clause.

* we can pass multiple argument to the group by clause but it will group the Record in combination.

order of execution

nature of execution.

① From

goes to the database search table put the table under execution.

② where

-It execute Row by Row

③ group by

Row by Row and creates groups

④ select

group by group execution.

Q:- WATD number of employees working in Each dept?

③ - select count(*), deptno

① - from emp

② - group by deptno;

① output of from clause.

Entire emp table

② output of where clause

no o/p as there is no where clause

③ output of Group By clause.

ENAME	JOB	SAL	DEPTNO
SMITH	CLERK	800	20
JONES	MANAGER	2975	20
SCOTT	ANALYST	3000	20
ADAMS	CLERK	1100	20
FORD	ANALYST	3000	20

ENAME	JOB	SAL	DEPTNO
ALLEN	SALESMAN	1600	30
WARD	SALESMAN	1250	30
MARTIN	SALESMAN	1250	30
BLAKE	MANAGER	2850	30
TURNER	SALESMAN	1500	30
JAMES	CLERK	950	30

ENAME	JOB	SAL	DEPTNO
CLARK	MANAGER	2450	10
KING	PRESIDENT	5000	10
MILLER	CLERK	1300	10

Output of select clause.

COUNT (*)	DEPTNO
5	20
6	30
3	10

Q:- WRTN Total no. of employee's working in each JOB.
except For Analyst

W select count(*), JOB
From emp
where Job != 'Analyst' OR Job NOT IN Analyst
group by JOB;

1) output of from clause

2) output of where clause.

Entire emp table

ENAME	JOB	SAL	DEPTNO
SMITH	CLERK	800	20
ALLEN	SALESMAN	1600	30
WARD	SALESMAN	1250	30
JONES	MANAGER	2975	20
MARTIN	SALESMAN	1250	30
BLAKE	MANAGER	2850	30
CLAKE	MANAGER	2450	10
KING	PRESIDENT	5000	10
TURNER	SALESMAN	1500	30
ADAMS	CLERK	1100	20
JAMES	CLERK	950	30
MILLER	CLERK	1300	10

3) output of gr

ENAME
SMITH
ADAMS
JAMES
MILLER

4)

ENAME
ALLEN
WARD
MARTIN
TURNER

ENAME
JONES
BLAKE
CLAKE

ENAME
KING

4) output

3) output of group by clause.

ENAME	JOB	SAL	DEPTNO
SMITH	CLERK	800	20
ADAMS	CLERK	1100	20
JAMES	CLERK	950	30
MILLER	CLERK	1300	10

4)

ENAME	JOB	SAL	DEPT NO.
ALLEN	SALESMAN	1600	30
WARD	SALESMAN	1250	20
MARTIN	SALES MAN	1250	30
TURNER	SALES MAN	1500	30

ENAME	JOB	SAL	DEPTNO.
JONES	MANAGER	2975	20
BLAKE	MANAGER	2850	30
CLARK	MANAGER	2450	10

ENAME	JOB	SAL	DEPTNO.
KING	PRESIDENT	5000	10

4) output of select clause.

Count(*)	JOB
4	CLERK
4	SALESMAN
3	MANAGER
1	PRESIDENT

Notes Rule ☹️

- we can display only group by column or expression along with any multiple row function in select clause.

Q: WAQT The maximum salary In each dept.

✓
Select max(SAL), deptno.
From emp.
group by deptno;

Q: WAQT the total salary needed to pay In each Job.
except for they employees who work in Dept no

20
✓
Select sum(SAL), job
From emp.
where dept != 20
group by job;

Q: WAQT The total no of employees whose Name start's with 'S' in each dept.

✓
Select ~~sum~~ (ENAME), deptno OR COUNT(*)
From emp.
where ~~NAME~~ ENAME LIKE 'S%'
group by deptno;

Q: WAQT The average salary needed to pay the employees working in who are hired in month of Feb in each Job.

✓
Select avg(SAL), job
From emp
where HIREDATE LIKE '%Feb%'
group by job;

Q: WAQT The mo
employees wh

Imp
Q: WAQT Nu
earning

Q: WAQT
more

* wrong
Q: WAQT
if
dep

Q: WAQTD The maximum salary in each job except for the employees who does not have MANAGER.

~~Ans who got have manager~~
~~select max(sal), job~~

~~from emp~~

~~where job != 'MANAGER'~~

~~group by job;~~

select max(sal), job

from emp

where mgr is not null

group by job;

Imp

Q: WAQTD Number of employee's working in some department and earning some salary.

→

select count(*), deptno, sal

from emp

group by deptno, sal;

Q: WAQTD Total salary of the employees if the employee earn's more than 1250 in is each designation.

select sum(sal), job

from emp

where sal > 1250

group by job;

X wrong
answer

Q: WAQTD The number of employees working in each dept. if there are at least 2 employee's working in each department

select count(*), deptno

from emp

where

group by deptno;

X Having clause

- It is use to filter the group.
- It executes after the execution of group of clause.
- It executes group by group.
- In group by clause we can write group filter condition Having
- we can pass multiple condition in Having clause by using logical operator.
- we can pass multirow function in Having clause

Syntax:-

```
SELECT group_function / group by - col / group by expression  
FROM table-name  
[ WHERE < filter-condition > ]  
GROUP BY col-name / expression  
HAVING < group-filter-condition >
```

* ORDER of Execution :-

- 1 - FROM
- 2 - WHERE --> Row-By-Row (if needed)
- 3 - GROUP BY --> Row-By-Row
- 4 - HAVING --> Group-By-Group
- 5 - SELECT.

Q:- WAGTD No. of employees working in each dept if there at least two employees working in each dept.

- ④ SELECT COUNT (*) IDNO
- ① FROM emp
- ② GROUP BY DNO
- ③ HAVING COUNT (*) >= 2;

group-filter-condition.

Emp

DATABASE
o/p of
from
clause.

EID	ENAME	ON0
1	T-HEAD	20
2	DHONI	10
3	RAJNA	30
4	GAYLE	30
5	VIRAT	30
6	BOMRAH	10
7	ROHIT	10

o/p of Group by clause.

20

1	T-HEAD	20
---	--------	----

1

$17 = 2 (F)$

10

2	DHONI	10
6	BOMRAH	10
7	ROHIT	10

3

$37 = 2 (T)$

30

3	RAJNA	30
4	GAYLE	30
5	VIRAT	30

3

$37 = 2 (T)$

o/p of Having clause.

count (*) > 2

17

o/p of Select clause.

count(*)	ON0
3	10
3	30

Q:- WAGTO designation of the employee's In which There are
Atleast 2 employees Working In each JOB ,

```
select JOB
from emp
group by JOB
having count(*) >= 2; or > 1
```

Q:- WAGTO NAMES THAT ARE REPEATED EXACTLY TWICE.

```
select ENAME
from emp
group by ENAME
having count(*) = 2;
```

Q:- WAGTO SALARIES THAT ARE REPEATED.

```
select SAL
from emp
group by SAL
having count(*) > 1;
```

Q:- WAGTO Number of employees working In each dept Having
Atleast 2 employee's CHARACTER 'A' OR 'S' In their NAMEs.

```
select count(*), deptno
from emp
group by deptno
having count(*) LIKE 'AS%' = 2;
```

```
select count(*) , deptno
from emp
where ENAME like '%A%' or
ENAME like '%S%'
group by deptno
having count(*) > 1;
```

010	(111-100)
01	2
08	0

H.W

Q:- W@TO TOTAL SALARY OF EACH JOB IF TOTAL SALARY OF EACH JOB IS GREATER THAN 3450.

→ ~~select sum(sal) as Job~~ ~~from emp~~ ~~where sal > 3450~~ ~~group by Job;~~
select sum(sal) as Job
from emp
group by Job
having sum(sal) > 3450;

Q:- W@TO Number of employees in each job and Total Salary of the employee's if they ARE EARNING SALARY more than 1500.

→ select count(*) as Job, sum(sal)
from emp
where sal > 1500
group by Job;

Total salary, minimum salary, Average salary then Having clause is used.

Q:- W@TO deptno and number of clerk working in each dept if There are Atleast 2 clerks in each dept.

→ select count(*) as deptno
from emp
where job = 'CLERK'
group by deptno
having count(*) >= 2;

Q:- W@TO deptno and Total Salary needed ToPAY ALL The employees in each dept if there are at least 4 employees in each dept.

→ select sum(sal) as deptno
from emp
group by deptno
having count(*) >= 4; OR >= 3

Q:- W@TO number of employees earning salary more than 1200 in each job And Total Salary needed ToPAY employees of each job must Excess 3800.

select count(*) as Job, sum(sal)
from emp
where sal > 1200
group by Job
having sum(sal) > 3800;

Q:- WAQTO deptno And number of manager working only If there are 2 manager working In each dept As managers

→
 SELECT deptno, count(*)
 From emp
 where job = 'MANAGER'
 group by deptno
 having count(*) = 2;

OIP:- No Row selected.

Q:- WAQTO number of employees working In each dept If there are Atleast 2 Analyst In each dept

→
 select count(*) deptno
 From emp
 where job = 'Analyst'
 group by deptno
 having count(*) >= 2;

Q:- WAQTO deptno and Total Salary of each dept If there are Atleast 4 employees working In each dept

→
 select sum(sal) deptno
 From emp
 group by deptno
 having count(*) >= 4;

Q:- WAQTO number of employees working in each job If there are Atleast 2 employees.

→
 select count(*), job
 From emp
 group by job
 Having count(*) >= 2;

Q:-

Q:- WAQTO Job wise maximum salaries If the maximum salary of each job is more than 1500

→
✓

```
select max(sal), job  
from emp  
group by job  
having max(sal) > 1500;
```

Q:- WAQTO Deptno And Average Salaries given to each dept If the Average salary is less than 3000

→
✓

```
select avg(sal), deptno  
from emp  
group by deptno  
having avg(sal) < 3000;
```

Q:- WAQTO number of employee's working in each dept If there are At least one manager in each dept.

→
✓

```
select count(*), deptno  
from emp  
where job = 'MANAGER'  
group by deptno  
having count(*) >= 1; OR > 0;
```

Q:- WAQTO The names which are present more than once OR Repeated or duplicated.

→
✓

```
select ENAME, count(*)  
from emp  
group by ENAME  
HAVING COUNT(*) > 1; OR > 2
```

Q:- WAQTO minimum salary of each dept excluding the president the minimum salary HAS TO BE MORE than 10000

→
✓

```
select min(sal), deptno  
from emp  
where job != 'PRESIDENT'  
group by deptno  
having min(sal) > 10000;
```


Q:- WAQTO Average Salary given To each Job By excluding the employees whose name ends with 'S' and the Average Salary must be less than 1500.

→
 select avg(SAL), Job
 from emp
 where ENAME NOT LIKE '%S'
 group by Job;
 having avg(SAL) < 1500;

V.VIMP

★ Difference between where clause & HAVING Clause.

where clause.	HAVING Clause.
1) It is use to filter the condition (Records).	1) It is use to filter the groups.
2) It executed Row by Row.	2) It executed group by group.
3) It execute's After execution of from clause	3) It executed After the execution of the Group By clause
4) In where clause we can not pass MRF (Multi Row Function)	4) In Having clause we can pass MRF (Multi Row Function)

Q:- WAQTO name, Salary of the employee's If the employee earning more than 2000.

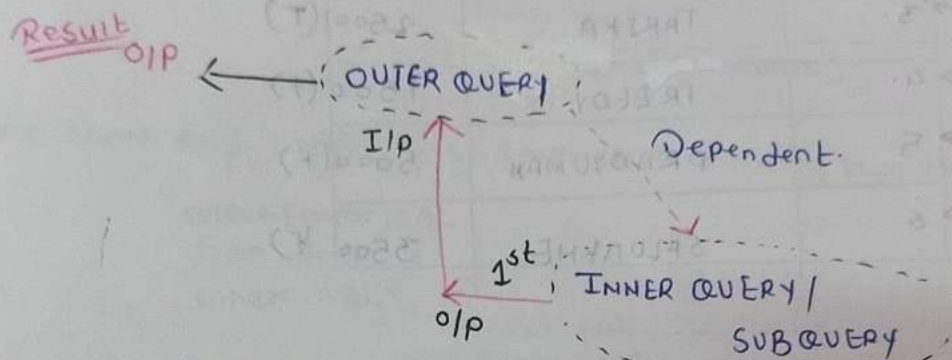
→
 select Ename, Salary
 from emp
 where SAL > 2000;

Q:- WAQTO name of the employee's If the employees are earning more than TARIKA.

→
 select Ename
 from emp
 where SAL >

☆ SUBQUERY ☆

- A query written inside another query is known as Subquery.
- Working Procedure or principle.
- Lets us consider two query's outer query and Inner query of Subquery.



- OUTER query
- Inner query / Subquery

- Inner query execute first (1st) and generate's the outputs.
- The outputs of the INNER query is passed Input As an outer query
- After that outer query executes and generate's the output
- The output of the outer query is known as Result.
- By This we can say outer query is dependent on Inner query
- Inner query close within CD parenthesis.
- WHEN OR WHY use Subquery

Case 1:-

Whenever we have unknown's then we use of concept of subquery

Emp

	EID	ENAME	SAL
→ 1.		DAYA	3000 (F)
→ 2.		ABHIJEET	3000 (F)
→ 3.		TARIKA	2500 (T)
→ 4.		FREEDY	1500 (f)
→ 5.		PRADYUMAN	5000 (F)
→ 6.		SALUNKHE	3500 (F)

Q:- WAQTD name of the employee's if the employee's are earning more than TARIKA

⑥ SELECT ENAME

④ FROM EMP

⑤ WHERE SAL >

③ SELECT SAL

① FROM EMP

② WHERE ENAME = 'TARIKA';

(T) 3000 > 2500

(T) 3000 > 2500

X (F) 2500 > 2500

X (F) 1500 > 2500

(T) 5000 > 2500

(T) 3500 > 2500

Result

o/p
DAYA
ABHIJEET
PRADYUMAN
SALUNKHE

Q:- WA@TD Name of the employees earning more than Adams.

```
Select Ename, SAL
```

```
From emp
```

```
where SAL >
```

```
(Select SAL
```

```
From emp
```

```
where Ename = 'Adams');
```

Q:- WA@TD name And salary of the employee's earning less than King.

```
select Ename, SAL
```

```
From emp
```

```
where SAL <
```

```
(Select SAL
```

```
From emp
```

```
where Ename = 'KING');
```

Q:- WA@TD name and deptno of the employee's if they are working in the same department as Jones

```
Select Ename, Deptno
```

```
From emp
```

```
where deptno = (Select deptno
```

```
From emp
```

```
where Ename = 'JONES');
```

Q:- WA@TD NAME and JOB of All the employees working in the same Designation as James.

```
Select Ename, JOB
```

```
From emp
```

```
where JOB = (Select JOB
```

```
From emp
```

```
where Ename = 'JAMES');
```

Q:- WA@TD EmpNo and ename Along with Annual SAL of All the employee's if their Annual salary is greater than WA@TD Annual Salary.

Select EMPNO, ENAME, SAL*12 AS Annual-Salary.
From emp

where Annual-SAL > (Select SAL*12*Annual-Sal
From emp
where ENAME = 'WARD');

Assignment

Q:- WAQTO name, salary, HIREDATE If the employees are earning more than ADAMS But HIRE Before MILLER

Select ENAME, SAL, HIREDATE
From emp
where SAL > (Select SAL
From emp
where ENAME = 'ADAMS')
AND HIREDATE < (Select HIREDATE
From emp
where ENAME = 'MILLER');

Q:- WAQTO All the details of the employees If the employees working in same designation as martin and earn between 1000 to 2500

Select *
From emp
where JOB = (Select JOB
From emp
where ENAME = 'MARTIN')
AND SAL BETWEEN 1000 AND 2500;

OR
Select *
From emp
where JOB SAL BETWEEN 1000 AND 2500 AND

Q:- WAQTO All the details of the employee If the employee's work in dept 20 or 30 and HIRE Before King

4.6.2
 Select *
 From emp
 where deptno = (20,30) AND
 HIREDATE < (select HIREDATE
 From emp
 where ENAME = 'KING')

Q:- WAOTD NAME AND HIREDATE of the employees if they are hired
 before TURNER.

→
 Select ENAME, HIREDATE
 From emp
 where HIREDATE < (select HIREDATE
 From emp
 where ENAME = 'TURNER');

Q:- WAOTD NAME AND HIREDATE of the employees if they are
 HIREO after the PRESIDENT.

Select ENAME, HIREDATE
 From emp
 where HIREDATE > (select HIREDATE
 From emp
 where ENAME = 'PRESIDENT');

Q:- WAOTD NAME AND SAL of the employees if they are earning
 SAL less than the employee whos empno is 7839.

Select ENAME, SAL
 From emp
 where SAL < (select SAL
 From emp
 where EMPNO = 7839);

Q:- WAQTD All the details of the employee if the employees are HIRED BEFORE MILLER.

→
Select *
From emp
where HIREDATE < (select HIREDATE
From emp
where ENAME = 'MILLER');

Q:- WAQTD ENAME and Empno of the employees if employees are earning more than ALLEN.

→
Select ENAME, Empno
From emp
where SAL > (select SAL
From emp
where ENAME = 'ALLEN');

Q:- WAQTD number of employees HIRED AFTER KING.

✓
Select ENAME, HIREDATE count(*)
From emp
where HIREDATE > (select HIREDATE
From emp
where ENAME = 'KING');

Q:- WAQTD Total SALARY given to the employee's working in the SAME Dept AS WARD.

✓
Select sum(SAL)
From emp
where DEPTNO = (select deptno
From emp
where ENAME = 'WARD');

Q:- WAO TO DNAME of SCOTT

```
DEPTNO  
select DNAME  
from DEPT  
where DEPTNO = (select DEPTNO  
from EMP  
where ENAME = 'SCOTT');
```

Q:- WAO TO DNAME of JONES:

```
select DEPTNO, DNAME  
from DEPT  
where DEPTNO = (select DEPTNO  
from EMP  
where ENAME = 'SCOTT');
```

Q:- WAO TO DEPTNO of Jones

```
select DEPTNO  
from EMP  
where ENAME = 'JONES';
```

Q:- WAO TO LOC of Smith.

```
select LOC  
from DEPT  
where DEPTNO = (select DEPTNO  
from EMP  
where ENAME = 'SMITH');
```

Q:- WAO TO DNAME of President.

```
select DNAME  
from DEPT  
where DEPTNO = (select DEPTNO  
from EMP  
where JOB = 'PRESIDENT');
```

Q:- WAO TO LOC of the employee's whose NAME ends with character 'T'

```
select LOC  
from DEPT  
where DEPTNO = (select DEPTNO  
from EMP  
where ENAME like '%T');
```


Q: WAO TO details of the employee working in Accounting Dept.

```

Select *
From emp
where deptno = (select deptno
                from dept
                where dname = 'Accounting');

```

Q: WAO TO Details of employees if the employees earn's more than 1500 AND work in Accounting dept.

```

Select *
From emp
where sal > 1500
AND deptno = (select deptno
              from dept
              where dname = 'Accounting');

```

Q: WAO TO Details of the employees if the employee's HIRE AFTER Allen and earn's between 1250 to 3000 work in the LOC of the DALLAS

```

Select *
From emp
where
AND HIREDATE > (select HIREDATE
                from emp
                where ename = 'Allen')
AND SAL BETWEEN 1250 AND 3000 AND
deptno = (select deptno
          from dept
          where loc = 'DALLAS');

```

Case 2:- where
executed o
of concept

Emp

EID	ENAME
1	ALLEN
2	MARTIN
3	MILLER
4	SCOTT
5	KING

Q: WAO TO

Q: WAO TO

Q: WAO TO

Subquery

Case 2^o - whenever the data to be selected and condition to be executed are presenting different table then we used of concept of subquery.

Emp

EID	ENAME	DNO	
1	ALLEN	20	(F)
2	MARTIN	10	(F)
3	MILLER	10	(F)
4	SCOTT	20	(T)
5	KING	30	(F)

DEPT

DNO	DNAME	LOC
10	D1	L1
20	D2	L2
30	D3	L3

Q: WQTD DNAME OF SCOTT:

⑥ SELECT DNAME
 ④ FROM DEPT
 ⑤ WHERE DNO = (SELECT DNO
 ① FROM EMP
 ② WHERE ENAME = 'SCOTT');

Q: WQTD LOC OF SMITH.

SELECT LOC
 FROM DEPT
 WHERE DNO = (SELECT DEPTNO
 FROM EMP
 WHERE ENAME = 'SMITH');

Q: WQTD DNAME OF PRESIDENT.

SELECT DNAME
 FROM DEPT
 WHERE DEPTNO = (SELECT DEPTNO
 FROM EMP
 WHERE JOB = 'PRESIDENT');

Q: WAO TO LOC OF THE EMPLOYEE'S WHOSE NAME ENDS WITH CHARACTER 'G'.

```
SELECT LOC  
FROM DEPT  
WHERE DEPTNO = CSELECT DEPTNO  
FROM EMP  
WHERE ENAME LIKE '%G';
```

Q: WAO TO DETAILS OF THE EMPLOYEES WORKING IN ACCOUNTING DEPT.

```
SELECT *  
FROM EMP  
WHERE DEPTNO = CSELECT DEPTNO  
FROM DEPT  
WHERE DNAME = 'Accounting';
```

Q: WAO TO NUMBER OF EMPLOYEES WORKING SALES DEPT.

```
SELECT COUNT(*)  
FROM EMP  
WHERE DEPTNO = CSELECT DEPTNO  
FROM DEPT  
WHERE DNAME = 'Sales';
```

* Notes

- The column name of single
- In inner query we can select only one column or expression
- In where clause of outer query and select clause of inner query column need not to be same but datatype must be identical

assignment no. 1:-

Q1) WQTD details of the employees earning more than 2000 in Accounting dept.

```
select *
from emp
where SAL > 2000
AND DEPTNO = (select DEPTNO
from dept
where DNAME = 'Accounting');
```

Q2) WQTD number of employees working AS a clerk in Research dept.

→

```
select count(*)
from emp
where JOB = 'CLERK' AND DEPTNO = (select DEPTNO
from dept
where DNAME = 'RESEARCH');
```

Q:- WQTD details of the employees HIRED AFTER SMITH into Accounting DEPT.

```
select *
from emp
where hiredate > (select hiredate
from emp
where ename = 'Smith')
AND deptno = (select deptno
from dept
where dname = 'Accounting');
```

Q WQTD Details of the employees working AS a manager earning less than King in the location of the new york.

```
select *
from emp
where JOB = 'manager'
AND SAL < (select sal
from emp
where ENAME = 'KING')
AND deptno = (select deptno
from dept
where loc = 'NEW YORK');
```


Q:- WAGTD numbees of employee's whose name start with character 'A' working in the same designation AS JONES HIRGD AFTER JONES eaming more than SMITH AND working in the location of CHICAGO.

→

```

select count(*)
from emp
where ename like 'A%'
AND Job = (select Job
            from emp
            where ename = 'Jones')
AND HIRGD > (select hiredate
             from emp
             where ename = 'Jones')
AND SAL > (select SAL
            from emp
            where ename = 'SMITH')
AND deptno = (select deptno
              from dept
              where loc = 'CHICAGO')

```

① Sing
It
Kn
In
well

EMP

EIO	
1	
2	
3	
4	
5	
6	
7	

Q:- WAG

✱ mu

ins

Types of subquery

- ① Single Row subquery → But table =
- ② Multi Row subquery.

① Single Row subquery.

If the subquery returns exactly one output or value, it's known as single row subquery.

In single row subquery, we can use equal operators as well as IN operator to compare.

EMP

EMPID	ENAME	JOB	DNO
1	Allen	clerk	20
2	King	President	10
3	Miller	clerk	20
4	Scott	manager	30
5	Jones	salesman	30
6	Martin	manager	10
7	Smith	manager	20

DEPT.

DNO	DNAME	LOC
10	D1	L1
20	D2	L2
30	D3	L3

Q: WHAT IS DNAME of President?

SELECT DNAME
FROM DEPT
WHERE DNO = (SELECT DNO
FROM EMP
WHERE JOB = 'PRESIDENT');

② Multi Row sub-query:

If the subquery returns more than one output or value, it's known as multi-row subquery.

- In multi row subquery, we cannot use equal operator instead we have to use IN operator to compare.

आइए multi-row sub query

इस = operator
not used.

Q:- WAOTD DNAME of manager?

✓
SELECT DNAME
FROM DEPT

WHERE DNO IN (SELECT DNO

FROM emp
WHERE JOB = 'MANAGER');

10 IN (20, 10)

20 IN (20, 10)

30 IN (20, 10)

✓ MRSQ

NOTES:- It's difficult to identify whether subquery belongs to single row subquery or multi row subquery therefore always use IN operator to compare.

Q:- WAOTD maximum sal of the employees if the employees earning less than King and work as manager or clerk in each dept.

SELECT max(SAL), DNO

FROM emp

WHERE SAL < (SELECT SAL
FROM emp
WHERE ENAME = 'KING')

AND JOB IN ('MANAGER', 'CLERK')

GROUP BY DNO;

Q:- WAOTD details of the employees if the employees earning less than maximum SAL of DEPTNO 10.

SELECT *

FROM emp

WHERE SAL < (SELECT max(SAL)
FROM emp

WHERE DEPTNO = 10);

Q:- WAOTD details of the employees if the employees hired after Allen and work in New York and works in some designation as miller and the employees earn less than Avg SAL of DEPT 20.

Select x

From emp

where HIREDATE > (SELECT HIREDATE

From emp

where ename = 'Allen') AND

DEPTNO = (SELECT DEPTNO

From DEPT

where LOC = 'NEWYORK')

AND JOB = (SELECT JOB

From emp

where ename = 'Miller')

AND SAL < (SELECT AVG(SAL)

From emp

where DEPTNO = 20);

*Subquery operator *

✓ ① ALL

✓ ② Any

ALL: ① It's a special operator which has to be used along with relational operator

② It returns true if it satisfied all the values at RHS.

EID	ENAME	JOB	SAL
1	OHOLU	SALES	950
2	BHOLU	CLERK	1200
3	KALIA	Manager	2000
4	TUNTUN	president	5000
5	KIRMA DA	manager	1100

Q: WAO TO name of the employees who earns less than managers

Select Ename

From emp

where SAL < ALL (select sal

From emp

where Job = 'MANAGER');

* ANY: It's a special operator which has to be used along with relational operators. It returns true if it satisfied any one of the values of RHS.

Q:- WAOTD name of the employees who earn's less than any one of the manager's

```
SELECT ENAME
FROM emp
WHERE SAL < ANY (SELECT SAL
                  FROM Emp
                  WHERE JOB = 'MANAGER');
```

output

Dholu
Bholu
Kirmoda

notes:- All of ANY operator has to be used along with relational operators

Q:- WAOTD name & SAL for all the employees if they are earning less than atleast a manager's

→

```
SELECT ENAME, SAL
FROM Emp
WHERE SAL < ANY (SELECT SAL
                  FROM Emp
                  WHERE JOB = 'MANAGER');
```

Q:- WAOTD name & hire date of employee hired before all the manager's

```
SELECT ENAME, HIREDATE
FROM Emp
WHERE HIREDATE < ALL (SELECT HIREDATE
                      FROM Emp
                      WHERE JOB = 'MANAGER');
```

Q:- WAQT name of the employees hired after all the manager & earning salary more than all the clerk.

```
SELECT ENAME,  
FROM emp  
WHERE HIREDATE > ALL (SELECT HIREDATE  
FROM emp  
WHERE JOB = 'MANAGER')  
AND SAL > ALL (SELECT SAL  
FROM emp  
WHERE JOB = 'CLERK');
```

Q:- WAQT details of the employees working as clerk hired before at least a salesman.

```
SELECT *  
FROM emp  
WHERE JOB = 'CLERK'  
AND HIREDATE < ANY (SELECT HIREDATE  
FROM emp  
WHERE JOB = 'SALESMAN');
```

Q:- WAQT details of the employee's working in Accounting or Sales dept.

```
SELECT *  
FROM emp  
WHERE DEPTNO IN (SELECT DEPTNO  
FROM DEPT  
WHERE DNAME IN ('ACCOUNTING', 'SALES'));
```

Q:- WAQT Dept name of the employees whose name is Smith, King & Miller.

```
SELECT DNAME  
FROM DEPT  
WHERE DEPTNO IN (SELECT DEPTNO  
FROM emp  
WHERE ENAME IN ('SMITH', 'KING', 'MILLER'));
```


Q:- WAO TO details of the employees working in New York or Chicago.

```
SELECT *
FROM Emp
WHERE DEPTNO IN (SELECT DEPTNO
                  FROM DEPT
                  WHERE LOC IN ('New York', 'CHICAGO'))
```

Q:- WAO TO employees names if emp are hired after all the employee of dept 10.

→

```
SELECT ENAME
FROM Emp
WHERE HIREDATE > ALL (SELECT HIREDATE
                       FROM Emp
                       WHERE DEPTNO = 10);
```

Q:- WAO TO maximum salary of the employees.

→

```
SELECT maxCSAL
FROM emp;
```

Q:- WAO TO the name of the employees who earns maximum salary.

```
SELECT ENAME
FROM Emp
WHERE SAL = (SELECT maxCSAL
              FROM Emp);
```

18/03/25

Q:- WAO TO 4th maximum salary.

⇒

18/03/25
Q:- WAO TO 4th

Q:- Nested Sub
A Subquery
Nest

- Innermost
we c

Emp	SAL
1250	
3000	
5000	
3000	
4500	
1250	
890	

Q:- WAO TO 4th

SELECT
FROM
W
1250 < 3000
3000 < 5000
5000 < 3000
3000 < 4500
4500 < 1250
1250 < 890

08/03/25

Q:- WATD 4th maximum salary.

Q:- Nested subquery (*)

A subquery written inside the another subquery is known as Nested subquery.

- Innermost subquery starts the execution we can next about 2nd subquery.

Emp	SAL
1250	4th max
3000	3rd max
5000	1st max
3000	3rd max
4500	2nd max
1250	4th max
890	5th max

Q:- WATD 4th maximum salary.

SELECT max(SAL)

From Emp

where SAL < (SELECT max(SAL)

1250 < 3000 T

3000 < 3000 F

5000 < 3000 F

3000 < 3000 F

4500 < 3000 F

1250 < 3000 T

890 < 3000 T

From emp

where SAL < (SELECT max(SAL)

From Emp

1250 < 4500 T

3000 < 4500 T

5000 < 4500 F

3000 < 4500 T

4500 < 4500 F

1250 < 4500 T

890 < 4500 T

where SAL < (SELECT max(SAL)

From emp)))

3000 < 5000 T

5000 < 5000 F

3000 < 5000 T

4500 < 5000 T

1250 < 5000 T

890 < 5000 T

24150130

Q:- WACOTD der

SELECT
FROM
WHERE

Q:- WACTD 10

SELE
Fro
whe

7 where
clause is
written

Q: WATO E

SEL
Fro
wt

Where SAL? CSELECT MIN(SAL)
From Emp
WHERE SAL?

Q:- WASTO dept details of the employee who is earning end max salary. (2)

SELECT *

From emp dept

where deptno IN (SELECT deptno

From Emp

where SAL IN (SELECT MAX(SAL)

From emp

where $SAL < CSELECT MAT(SAL)$

From Emp)))

Q: WABTD location of the employee who is earning 5th minimum salary.

→

SELECT LOC

From dept

where deptno in (select deptno

From Emp

where $SAL_{in} = (SELECT \min(SAL))$

From emp

WHERE SALY (SELECT min(SALY

From emp

WHERE SAL > CSAL69

 $\min_{\mathbf{C}} \|\mathbf{C}\|_F$

Frame mp

(continued)

7 where
clause is
written.

Q: WATO employee details who earn within salary.

SELECT *

From emp

where $SALIN = CSELECT \min(SAL)$

From Emp

where $SAL < (\text{select min(say)})$

From emp

WHERE SAL < (SELECT min(SAL))

From emp

where SAL < (SELECT min(SAL))

From crop :

۱۷۷۵

1913125

employee manager Relation

case 1: To find reporting manager (EID = MGR)

Q:- WAQTD name of Smith's Reporting manager.

⑥ SELECT ENAME
⑥ From Emp
⑥ Where EID = (SELECT MGR
① From Emp
② Where ENAME = 'SMITH');

1 = 5 (F)
2 = 5 (F)
3 = 5 (F)
4 = 5 (F)
5 = 5 (T)
6 = 5 (F)

EID	ENAME	MGR	
1	ALLEN	2	F
2	MILLER	3	F
3	SCOTT	5	F
4	JONES	2	F
5	KING	NULL	F
6	SMITH	5	T

Q:- WAQTD miller's manager's manager name.

SELECT ENAME
From Emp
Where EID = (SELECT MGR
From Emp
Where EID = (SELECT MGR
From Emp
Where ENAME = 'MILLER'));

Q:- WAQTD the location in which Smith's manager is working.

SELECT LOC
From DEPT
Where DEPTNO = (SELECT DEPTNO
From Emp
Where EID = (SELECT MGR
From Emp
Where ENAME = 'SMITH'));

Q:- WAQTD name of the employees who are reporting to King.

SELECT ENAME
From Emp
Where MGR IN (SELECT EMPNO
From Emp
Where ENAME = 'KING');

case 2:- To find employee mgr IN EMPNO)

Q:- WAQTD Name of the employees who are reporting to king.

SELECT ENAME

From Emp

Where mgr IN (SELECT EmpNO
FROM Emp

Where ENAME = 'KING');

SCOTT
SMITH

2 = 5 (F)

3 = 5 (F)

5 = 5 (T)

2 = 5 (F)

NOU = 5 (F)

5 = 5 (T)

Q:- WAQTD no. of employee who are reporting to blake.

SELECT COUNT(*)

From emp

Where mgr IN (SELECT EmpNO
FROM Emp

Where ENAME = 'BLAKE');

Q:- WAQTD deptname of the employees who are reporting KING.



SELECT DNAME

From DEPT

Where DEPT IN (SELECT DEPTNO

FROM DEPT Emp

Where mgr IN (SELECT EmpNO

FROM Emp

Where ENAME = 'KING');

subquery (Answer to be given in Interview)

- Defⁿ
- Working principle / procedure
- CASES
 - ↳ CASE 1 ✓
 - ↳ CASE 2 ✓
- Types of subquery
 - ↳ single ✓
 - ↳ multi ✓
- subquery operator
 - ↳ ALL ✓
 - ↳ ANY ✓
- Nested subquery.

Assignment:

Q:- WAOTD deptname of miller's manager.

```
SELECT DEPTNAME
FROM DEPT
WHERE DEPTNO = (SELECT DEPTNO
                FROM EMP
                WHERE EMPNO = (SELECT MGR
                              FROM EMP
                              WHERE ENAME = 'MILLER'))
```

Q:- WAOTD salary of Adam's manager.

```
SELECT SAL
FROM EMP
WHERE EMPNO = (SELECT MGR
               FROM EMP
               WHERE ENAME = 'ADAMS'))
```

⇒

From emp

where ENAME = 'JONES'));

SELECT DNAME

where: deptno in (select deptno
from emp

where ENAME = 'KING'));

→

From emp

where ENAME = 'BLAKE')

→

From DEPT

WHERE EMPNO IN (SELECT MGR
FROM EMP

where
ENAME = 'SMITH';

Case 1

Q:- WA@TD name of the employees who earn more than Allen.



Select ename

From emp

where sal > (select sal

From emp

where ename = 'Allen');

Q:- WA@TD name and salary of the employee earning less than King

Select ename, sal

From emp

where sal < (select sal

From emp

where ename = 'King');

Q:- WA@TD name and deptno of the employee if they are working in the same dept as Jones.

Select ename, deptno

From emp

where deptno = (select deptno

From emp

where ename = 'Jones');

Q:- WA@TD empno and ename along with Annual salary of all the employees if their Annual salary is greater than WARS Annual salary.

Select empno, ename, sal*12 As Annual-Salary

From emp

where Annual-sal sal*12 > (select sal*12

From emp

where ename = 'WARS');

Case 2

Q:- WA@TD DNAME of the employee's works as President.

Select DNAME

From dept

where Deptno = (select deptno

From emp

where DNAME = 'President');

Q: WHAT DNAME of manager?

```
select DNAME  
from dept  
where deptno in (select deptno  
from emp  
where job = 'manager');
```

Q: WHAT numbers of employees working in sales dept?

```
select count(*)  
from emp  
where deptno in (select deptno  
from dept  
where dname = 'sales');
```

Q: WHAT details of the employee earning more than 2000 in Accounting dept?

```
select *  
from emp  
where SAL > 2000
```

```
AND deptno in (select deptno  
from dept
```

Q: WHAT numbers of employee working as clerk in Research dept?

```
select count(*)  
from emp  
where job = 'clerk' AND  
deptno in (select deptno  
from dept
```

```
where dname = 'research');
```

Q: WHAT details of the employees HIRED AFTER SMITH and INTO Accounting dept?

```
select *  
from emp  
where deptno HIREDATE > (select HIREDATE  
from emp  
where NAME = 'SMITH')
```


AND deptno IN (select deptno
from dept
where dname = 'Accounting');

Q:- WAO TO details of the employee's working as manager
earning SAL less than KING IN THE LOCATION OF New York.

→

select *
from emp
where job = 'manager'
AND SAL < (select SAL
from emp
where ename = 'KING')

AND deptno IN (select deptno
from dept
where loc = 'New York');

Q:- WAO TO number of employee whose name starts with
CHARACTER 'A' working IN THE SAME designation AS James
HIRED AFTER Jones earning more than Smith AND working
In the location of CHICAGO.

select count(*)
from emp
where ENAME LIKE 'A%' AND
JOB IN (select JOB
from emp
where ENAME = 'James')
AND HIREDATE > (select HIREDATE

from emp
where ename = 'Jones')
AND SAL > (select SAL
from emp
where ENAME = 'Smith') AND
deptno IN (select deptno
from dept
where loc = 'CHICAGO')

Q:- WAO TO names of the employees more than President

select ename
from emp
where SAL > (select SAL
from emp
where job = 'president');

Q:- WAO TO names

select
from
where

→ Subquery operator

- ① ALL
- ② Any

'ALL':- ALL oper
with Relat

② All operat
satisfied th

ENO	ENAME
①	OGGY
②	JACK
③	BOB
④	OLL-I
⑤	JABLU
⑥	TAPLU

Any:- An
Glon

- Any oper

Q:- WAO TO
the ch

Q: WAO TO name's of the employees who earn more than clerk

select ~~enome~~ enome
from emp
where SAL > select SAL
from emp
where Job = 'clerk';

→ Subquery operators

- ① ALL
- ② Any

1) ALL:- ALL operator IS A special operators which Has to be used Along with Relational operators. To compare the values

2) All operator Return's true if all the value's At the RHS HAVE satisfied the condition

ENO	ename	Job	SAL
①	OGGUY	SALESMAN	1250
②	JACK	CLERK	950
③	BOB	PRESIDENT	5000
④	OLL-I	CLERK	1500
⑤	JABLU	MANAGER	3000
⑥	TAPLU	SALESMAN	1300

Q: WAO TO name of the employees who earning more than clerk

⇒ ⑥ select enome
④ from emp
⑤ where SAL > ALL (select SAL
① from emp
② where Job = 'clerk');

(F) 1250 > ALL (950, 1500)
(F) 950 > ALL (950, 1500)
(T) 5000 > ALL (950, 1500)
(F) 3000 > ALL (950, 1500)
(F) 1300 > ALL (950, 1500)
(F) 1500 > ALL (950, 1500)

OP:

950
1500

Any:- Any operator IS A special operator which has to be used along with Relational operator to compare the values

- Any operator's Return's True if Any one of the value's At the RHS HAVE SATISFIED THE condition.

Q: WAO TO name of the employee who earns more than any one of the clerk.

select enome
from emp
where SAL > Any (select SAL
from emp
where Job = 'clerk');

(T) 1250 > Any (950, 1500)
(F) 950 > Any (950, 1500)
5000 > Any (950, 1500)
3000 > Any (950, 1500)
1300 > Any (950, 1500)

OP:

950
1500

Notes:- All & any operators should be used Along with Relational operators

Q: WAOTD maximum salary of the employees

```
select max(sal)
from emp;
```

Q: WAOTD name of the employees who earn maximum salary.

```
select ename,
from emp
where sal = (select max(sal)
from emp);
```

Q: WAOTD 2nd maximum salary:

```
select max(sal)
from emp
where sal < (select max(sal)
from emp
where sal < (select max(sal)
from emp
where sal < (select max(sal)
from emp));
```

H.W

Q: WAOTD name of the employee earning 2nd minimum salary.

```
select ename
from emp
where sal > (select min(sal)
from emp
where sal > (select min(sal)
from emp));
```

Q: WAOTD dept name of the employee earning with minimum salary.

```
select deptname
from dept
from deptno in (select deptno
from emp
where sal = (select min(sal)
from emp
where sal > (select min(sal)
from emp
where sal > (select min(sal)
from emp));
```


From emp
where SAL > (select min(SAL)
From emp);

Q:- What is the location of the employee earning and maximum salary?

select LOC
From dept
where deptno in (select deptno
From emp
where SAL in (select max(SAL)

From emp
where SAL < (select max(SAL)
From emp);

Q:- What are the details of the employee earning and maximum salary?

select *
From dept
where deptno in (select deptno
From emp
where SAL in (select max(SAL)

From emp
where SAL < (select max(SAL)
From emp
where SAL < (select
max(SAL)
From emp);

Q:- What are the locations of the employees who are earning within salary?

select LOC
From dept
where deptno in (select deptno
From emp
where SAL in (select min(SAL)

From emp
where SAL > (select min(SAL)
From emp
where SAL > (select min(SAL)
From emp
where SAL > (select min(SAL)
From emp);

20/3/25

Single Row Functions

- | | |
|-------------|-------------|
| ① LENGTH() | ⑦ INSTR() |
| ② CONCAT() | ⑧ REPLACE() |
| ③ UPPER() | ⑨ REVERSE() |
| ④ LOWER() | ⑩ MOD() |
| ⑤ INITCAP() | ⑪ NVL() |
| ⑥ SUBSTR() | ⑫ ROUND() |
| | ⑬ TRUNC() |
| | ⑭ TO_CHAR() |

① LENGTH() :-

This function is used to count the number of characters present in the given string

Syntax:- LENGTH('string')

Ex:-

```
SELECT LENGTH ('ADITYA')
FROM DUAL;
```

 = 6.

Ex:-

```
SELECT LENGTH ('DHAWAL')
FROM DUAL;
```

 = 7

Q: Write name of the employees if the employee name have exactly four characters without using like operator.

```
SELECT ENAME
FROM Emp
WHERE LENGTH (ENAME) = 4;
```

* DUAL :-

It's a dummy table which has exactly one row and one column & which is used to get the results

① CONCAT :-
This Function is used to Join given two string

Syntax :-

CONCAT ('string 1', 'string 2')

Ex:- SELECT CONCAT ('ma', 'RUSHIKESH')
FROM DUAL;
⇒ MR. RUSHIKESH

② UPPER () :-

This Function is used to convert the given string into upper case.

Syntax :-

UPPER ('string')

Ex:- SELECT UPPER ('rupali')
FROM DUAL;
o/p:- RUPALI.

③ LOWER ()

This Function is used to convert the given string into lower case.

Syntax :-

LOWER ('string')

Ex:- SELECT LOWER ('SHYAM')
FROM DUAL;
⇒ shyam

④ INITCAP () :-

This Function is used to convert the initial Alphabets of a String To the UPPER case and rest of the alphabets into Lower case

Syntax:- INITCAP ('string')

Ex:- SELECT INITCAP ('mayursh')
FROM DUAL;
o/p:- Mayursh

⑤ SUBSTR () :-

It function is used to extract the part of the original string

Syntax:- SUBSTR('original-string', POSITION, LENGTH)

SUBSTR('MAHABHARAT', 1, 4) $\Rightarrow$ MAHA.

ive: ① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩

Ex:-

SUBSTR('MAHABHARAT', 1, 4) $\Rightarrow$ MAHA

① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩

① ② ③ ④

Ex:-

SUBSTR('MAHABHARAT', 6, 5) $\Rightarrow$ BHARAT

① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩

① ② ③ ④ ⑤

Ex-SUBSTR('MAHABHARAT', 2, 4) $\Rightarrow$ AHABHARAT.

Ex-SUBSTR('MAHABHARAT', 5, 7) $\Rightarrow$ BHARAT.

Ex-SUBSTR('MAHABHARAT', 2, 3) $\Rightarrow$ AHA.

Ex-SUBSTR('MAHABHARAT', 4, 1) $\Rightarrow$ A.

Ex-SUBSTR('MAHABHARAT', -6, 5) $\Rightarrow$ BHARA.

Ex-SUBSTR('MAHABHARAT', -10, 1) $\Rightarrow$ MAHABHARAT.

Ex-SUBSTR('MAHABHARAT', 5, 7) $\Rightarrow$ BHARAT.

Q:- WAO TO FIRST 3 CHARACTERS of Employees name.

SELECT SUBSTR('ENAME', 1, 3)
from emp;

Q:- WAO TO LAST 3 CHARACTERS of employees name.

SELECT SUBSTR('ENAME', -3)
from emp;

Q:- WAO TO The LAST CHARACTER of the employees NAME.

SELECT SUBSTR('ENAME', -1)
from emp;

Q:- WAO TO FIRST HALF of the employee names.

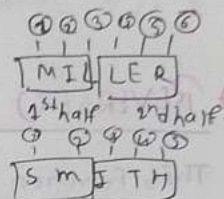
Q:- WAO TO SECOND HALF of the employee names.

① SELECT SUBSTR (ENAME, 1, LENGTH (ENAME) / 2)
From Emp;

② SELECT SUBSTR (ENAME, LENGTH (ENAME) / 2 + 1)
From Emp;

SUBSTR ('MILLER', 1, LENGTH ('MILLER') / 2)

SUBSTR ('SMITH', LENGTH ('SMITH') / 2)



Q:- WAO TO names of ALL The employees By Excluding the 1st and the LAST CHARACTER

→ SELECT SUBSTR (ENAME, 2, LENGTH (ENAME) - 2)
From emp;

Q:- WAO TO NAMES of the Employees if the second character of the name is 'L'

→ SELECT ENAME
From emp
where SUBSTR (ENAME, 2, 1) = 'L';

Q:- WAO TO some details of an employee if the 1st Three character of JOB is 'MAN'

→ ~~SELECT *~~
~~From emp~~
~~where JOB LIKE 'MAN%';~~

SELECT *
From emp
where SUBSTR (JOB, 1, 3) = 'MAN';

Q:- WAO TO NAME and HIREDATE of the employees HIRED IN THE YEAR 81.

SELECT ENAME, HIREDATE

From emp

where SUBSTR (HIREDATE, -2) = 81;

* REVERSE()

This Function is used to Reverse the given String

Syntax: REVERSE ('string')

Ex: SELECT REVERSE ('UTKARSH')
From DUAL; $\Rightarrow$ HSRAKTU

Ex: SELECT REVERSE ('BANANA')
From DUAL; $\Rightarrow$ ANANAB

* REPLACE()

This function is used to replace the given ^{old}String with new String or it will Remove the old String

Syntax: ('original_string', 'old_string' [new_string])

Ex: SELECT REPLACE ('BANANA', 'N', 'T')
From DUAL; $\Rightarrow$ BATATA

Ex: SELECT REPLACE ('BANANA', 'N') $\Rightarrow$ BAAA.

Q:- WAO TO count the number of character A present in the particular names;

~~SELECT count(*)
From emp
where~~

SELECT LENGTH (ENAME) - LENGTH (REPLACE (ENAME, 'A'))
From Emp.

Q:- WROTD name of the employees whose name have character
Exactly twice.

SELECT ENAME
From Emp
Where LENGTH (ENAME) - LENGTH (REPLACE (ENAME, 'A')) = 2;

* INSTR ()

This function is use to obtain the position of the string
which is present in the original string

Syntax: INSTR ('original-string', 'String', POSITION, [OCCURANCE])

-ve
INSTR ('MALAYALAM', 'A', 4, 2)
tve
1 2 3 4 5 6 7 8 9
x 1 x 2
⇒ 4

-9 -8 -7 -6 -5 -4 -3 -2 -1
INSTR ('MALAYALAM', 'A', 3, 3)
1 2 3 4 5 6 7 8 9
x 1 x 2 x 3
⇒ 8

INSTR ('MALAYALAM', 'A', 5, 7)
1 2 3 4 5 6 7 8 9
x 1 x 2
⇒ 0

INSTR ('MALAYALAM', 'A', 5, .)
1 2 3 4 5 6 7 8 9
x 1 x 2 x 3
⇒ 6

INSTR ('MALAYALAM', 'A', -3, 3)
5 6 7 8 9
x 1 x 2 x 3
⇒ 7

In position the value is always +2.
and position value if -3 then move LHS not RHS
make of length only

Q:- WHAT names of the employees who HAVE character 'A' IN THEIR NAME.

→

```
SELECT ENAME
FROM emp
WHERE INSTR(ENAME, 'A', 1) > 0;
```

Q:- WHAT names of the employee who HAVE CHARACTER 'L' ATLEAST TWICE IN THEIR NAME.

→

```
SELECT ENAME
FROM emp
```

```
WHERE INSTR(ENAME, 'L', 1, 2) > 0;
```

Q:- WHAT names of the employees who HAVE CHARACTER 'A' ATLEAST 3 times IN THEIR NAME.

→

```
SELECT ENAME
FROM emp
```

```
WHERE INSTR(ENAME, 'A', 1, 3) > 0;
```

* ROUND()

This function is used to Round off the given numbers To its nearest value.

Syntax: $\text{ROUND}(\text{NUMBER})$

Ex: $\text{ROUND}(7.2) = 7$

$\text{ROUND}(7.8) = 8$

$\text{ROUND}(7.5) = 8$

or write a query to display Round off the Average Sal of the employees.

SELECT ROUND(AVG(SAL))
FROM emp;

* TRUNC()

This function is used to Round off the given numbers To the lowest value values

Ex:

$\text{TRUNC}(7.9) = 7$

$\text{TRUNC}(7.5) = 7$

$\text{TRUNC}(7.1) = 7$

* MOD()

This function is used to obtain the Remainder of the given values

Syntax: $\text{MOD}(M, N)$

Ex: 1) SELECT MOD(4, 2)

FROM DUAL;

→ 0

2) SELECT MOD(5, 2)

(FROM DUAL);

→ 1

* NVL()

NULL VALUE Logic

This function is used to eliminate the side effects of Null in Arithmetic operation

Syntax: NVL(argument 1, argument 2)

EMP

Emp	ENO	ENAME	SAL	comm
1		A	800	200
2		B	1250	NULL
3		C	1500	0
4		D	2500	NULL
5		E	1920	20

Fig: Result Table

ENAME	SAL + NVL(comm, 0)
A	1000
B	1250
C	1500
D	2500
E	1940

Q: WAOB ENAME, Total earning of each employees

SELECT ENAME, SAL + NVL(comm, 0)
From Emp;

$$800 + 200 = 1000$$

$$1250 + 0 = 1250$$

$$1500 + 0 = 1500$$

$$2500 + 0 = 2500$$

$$1920 + 20 = 1940$$

* TO_CHAR() :-

This function is used to convert's the given date ^{into} String format by using format models.

Syntax: TO_CHAR(DATE, 'format-models')

format models

- | | | |
|-----------|---------|-----------|
| (1) YEAR | (5) MON | (9) D |
| (2) YYYY | (6) MM | (10) HH24 |
| (3) YY | (7) DD | (11) HH12 |
| (4) MONTH | (8) DAY | (12) MI |
| | | (13) SS |

Ex:- SELECT TO_CHAR(SYSDATE, 'YEAR:MONTH:DD:DAY:HH24:MI:SS')
FROM DUAL;

OIP:- TWENTY TWENTY-FIVE: MARCH

Ex:- SELECT TO_CHAR(SYSDATE, 'YY:MM:DAY:O:HH12:MI:SS')
FROM DUAL;

OIP:- 25:03:SATURDAY:7:04:10:37

Q:- WAOTD details of the employees hired in the month of feb.

SELECT *
FROM emp
WHERE TO_CHAR(HIREDATE, 'MM') = '02'.

Q:- WAOTD details of the employees who hired on wednesday.

SELECT *
FROM emp
WHERE TO_CHAR(HIREDATE, 'DAY') = 'Wednesday'

* Joins *

Joins: It is used to retrieve the data from multiple tables simultaneously.

There are five types in joins:-

- ① colesion join / cross join
- ② Innee join / equi join
- ③ outee join
 - ↳ Left outee join
 - ↳ Right outee join
 - ↳ Full outee join
- ④ Natural join
- ⑤ Self join

① Cartesian Join / cross Join

In cartesian join, the records of table 1 will merge with all records of table 2.

(Syntax:-

① ANSI (American national standard industry)

```
SELECT COLUMN-NAME
FROM TABLE-NAME.1 CROSS JOIN TABLE-NAME2;
```

② Oracle

```
SELECT COLUMN-NAME
FROM TABLE-NAME1, TABLE-NAME2;
```

EMP

DEPT

EMPID	ENAME	ON0	ON0	ONAME	LOC
1	A	20	10	D1	L1
2	B	30	20	D2	L2
3	C	10	30	D3	L3

```
SELECT *
FROM Emp, DEPT;
```

* Join *

Result Table

EMPID	ENAME	ON0	ON0	ONAME	LOC
1	A	20	10	D1	L1
1	A	20	20	D2	L2
1	A	20	30	D3	L3
2	B	30	10	D1	L1
2	B	30	20	D2	L2
2	B	30	30	D3	L3
3	C	10	10	D1	L1
3	C	10	20	D2	L2
3	C	10	30	D3	L3

In coelesion join we will get matching records along with plenty of unmatched records. The column or present in result table will be equals to the sum of column or present in both the tables.

The records or present in Result table will be equals to the product of records or present in both the tables.

* Inner Join / equi Join :-

It is used to obtain matching records.

Syntax:-

① ANSI (American national standard Institute)

```
SELECT COLUMN-NAME
FROM TABLE-NAME1, INNER JOIN TABLE-NAME2
ON <JOIN CONDITION>
```

② ORACLE

```
SELECT COLUMNNAME
FROM TABLE-NAME1, TABLENAME2
WHERE <JOIN CONDITION>
```

Join :

It is used to merge two tables.

Syntax:-

TABLE-NAME1.COLUMN-NAME = TABLE-NAME2.COLUMN-NAME

Emp

EmpID	ENAME	ONNO
1	A	20
2	B	10
3	C	20
4	D	30

DEPT.

DNO	Dname	LOC
10	D1	L1
20	D2	L2
30	D3	L3

Q:- WAO TO ENAME & DNAME

```
SELECT ENAME, DNAME
FROM Emp, DEPT
WHERE Emp.DNO = DEPT.DNO
```

$20 = 10$ (F) $10 = 10$ (T) $20 = 10$ (F) $30 = 10$ (F)
 $20 = 20$ (T) $10 = 20$ (F) $20 = 20$ (T) $30 = 20$ (F)
 $20 = 30$ (F) $10 = 30$ (F) $20 = 30$ (F) $30 = 30$ (T)

Result TABLE:

ENAME	DNAME
A	D2
B	D1
C	D2
D	D3

Q:- WAO TO employee details & dept details

```
SELECT *
FROM Emp, DEPT
WHERE Emp.DNO = DEPT.DNO;
where emp.deptno = Dept.DEPTNO;
```

Q:- WAO TO ENAME & salary along with their location if they are earning more than 2000.

```
SELECT ENAME, SAL, LOC
FROM Emp, DEPT
WHERE SAL > 2000 AND
Emp.DEPTNO = DEPT.DEPTNO;
```


Q:- WA@TD Ename & DEPTNO along with DEPTNAME & LOC if the employee is working as MANAGER.

```
SELECT Ename, Emp.DEPTNO, DNAME, LOC
FROM Emp, DEPT
WHERE Emp.DEPTNO = DEPT.DEPTNO
AND JOB = 'MANAGER';
```

Q:- WA@TD details of the employees along with deptname if the employee is earning more than 1000 & working in DeptNO -20.

```
SELECT Emp.*, DNAME
FROM Emp, DEPT
WHERE Emp.DEPTNO = DEPT.DEPTNO
AND SAL > 1000
AND Emp.DEPTNO = 20;
```

Q:- WA@TD Ename & DNAME of the employee who is working as MANAGER in LOC DALLAS.

```
SELECT Ename, DNAME
FROM Emp, DEPT
WHERE Emp.DEPTNO = DEPT.DEPTNO AND
JOB = 'MANAGER' AND
LOC = 'DALLAS';
```

Assignment:-

Q:- WA@TD name of the employee & location of all the employees.

```
SELECT Ename, LOC
FROM Emp, DEPT
WHERE Emp.DEPTNO = DEPT.DEPTNO;
```

Q:- WA@TD DNAME & salary for all the employee working in Accounting.

```
SELECT DNAME, SAL
FROM Emp, DEPT
WHERE Emp.DEPTNO = DEPT.DEPTNO
AND DNAME = 'ACCOUNTING';
```


Q:- WAQTD oname & Annual SAL for all the employee who's salary is more than 2340.

```
SELECT oname, SAL*12 AS ANNUAL_SAL
FROM emp, DEPT
WHERE Emp.DEPTNO = DEPT.DEPTNO AND
      SAL > 2340;
```

Q:- WAQTD ename & oname for employee having character 'A' in their oname.

```
SELECT ename, oname
FROM emp, DEPT
WHERE Emp.DEPTNO = DEPT.DEPTNO AND
      oname LIKE '%A%';
```

Q:- WAQTD ename & oname for all the employees working as sales man.

```
→ SELECT ename, oname
   FROM emp, DEPT
   WHERE Emp.DEPTNO = DEPT.DEPTNO AND
         JOB = 'SALESMAN';
```

Q:- WAQTD oname & job for all the employees who job oname starts with character 'S'

```
SELECT oname, JOB
FROM emp, DEPT
WHERE Emp.DEPTNO = DEPT.DEPTNO AND
      JOB LIKE 'S%' AND
      oname LIKE 'S%';
```

Q:- WAQTD oname & MGRNO for employees reporting to 7839.

```
SELECT oname, MGR
FROM emp, DEPT
WHERE Emp.DEPTNO = DEPTNO AND
      MGR = 7839;
```


Who's
Q:- WA@TD oName & HIREDATE for employee HIRED After 83
Into Accounting or Research Dept.

```
SELECT ONAME, HIREDATE
FROM Emp, DEPT
WHERE Emp.DEPTNO = DEPT.DEPTNO AND
      HIREDATE > '31-DEC-83' AND
      ONAME IN ('ACCOUNTING', 'RESEARCH');
```

Q:- WA@TD Ename & oName of the employees who are getting
comm in dept 10 or 30

```
SELECT Ename, oName
FROM Emp, DEPT
WHERE Emp.DEPTNO = DEPT.DEPTNO AND
      COMM IS NOT NULL AND
      DEPT.DEPTNO IN (10, 30);
```

Q:- WA@TD oName & empno for all the employees who is empno
or (7839, 7902) and are working in loc New York.

```
SELECT oName, Emp.EMPNO
FROM Emp, DEPT
WHERE Emp.DEPTNO = DEPT.DEPTNO
      AND Emp.EMPNO IN (7839, 7902)
      AND Loc = 'New York';
```

③ OUTER JOIN: It is used to obtain unmatched records along with
matching records.

It is used to obtain unmatched records along with matching
records.

There are three types in outer Join

① Left outer join:-

② Right outer join.

③ Full outer join.

① Left outer join:-

It is used to obtain unmatched records from the left table
along with matched records.

Syntax:-

① ANSI:-

SELECT COLUMN_NAME
FROM TABLE_NAME1 LEFT [OUTER] JOIN TABLE_NAME2
<JOIN CONDITION>;

② Oracle:-

SELECT COLUMN_NAME
FROM TABLE_NAME1, TABLE_NAME2
WHERE TABLE_NAME1.COL_NAME = TABLE_NAME2.COL_NAME
Join condition.

Emp

ENAME	DEPTNO
A	20
B	NULL
C	30
D	NULL

DEPT

DEPTNO	DNAME
10	D1
20	D2
30	D3
40	D4

SELECT *

FROM emp, DEPT

WHERE Emp.DEPTNO = DEPT.DEPTNO (+);

Join condition.

20 = 10 (F)

20 = 20 (T)

20 = 30 (F)

20 = 40 (F)

NULL = 10 (F)

NULL = 20 (F)

NULL = 30 (F)

NULL = 40 (F)

NULL

30 = 10 (F)

30 = 20 (F)

30 = 30 (T)

30 = 40 (F)

NULL = 10 (F)

NULL = 20 (F)

NULL = 30 (F)

NULL = 40 (F)

NULL

Result Table :-

ENAME	DEPTNO	DEPTNAME	DNAME	
A	20	20	D2	} matching Records
B	30	30	D3	
C	NULL	NULL	NULL	} unmatching Record of left table.
D	NULL	NULL	NULL	

② Right outer Join :-

It is used to obtain unmatched records from the right table along with matching records.

Syntax:-

① ANSI :-

```
SELECT COLUMN-NAME
FROM TABLE-NAME1 RIGHT [OUTER] JOIN TABLE-NAME2
ON <JOIN CONDITION>;
```

② Oracle :-

```
SELECT COLUMN-NAME
FROM TABLENAME1, TABLENAME2
WHERE TABLENAME1.COL-NAME (+) = TABLENAME2.COL-NAME
```

```
SELECT *
FROM EMP, DEPT
WHERE EMP.DEPTNO (+) = DEPT.DEPTNO;
```

Join conditions

$20 = 10$ (F)
 $20 = 20$ (T)
 $20 = 30$ (F)
 $20 = 40$ (F)

$NULL = 10$ (F)
 $NULL = 20$ (F)
 $NULL = 30$ (F)
 $NULL = 40$ (F)

NULL

$30 = 10$ (F)
 $30 = 20$ (F)
 $30 = 30$ (T)
 $30 = 40$ (F)

$NULL = 10$ (F)
 $NULL = 20$ (F)
 $NULL = 30$ (F)
 $NULL = 40$ (F)

NULL

Result:-

ENAME	DEPTNO	DEPTNO	ENAME	
A	20	20	D2	} matching Records
C	30	30	D3	
NULL	NULL	10	D1	} unmatched Records of Right table
NULL	NULL	40	D4	

③ Full outer join:-

It is used to obtain unmatched records from both the tables along with matching records.

Syntax:-

only ANSI:

SELECT COLUMN-NAME
 FROM TABLENAME1 FULL [OUTER] JOIN TABLE
 ON <Join condition>

select *
 from emp FULL JOIN dept
 on emp.DEPTNO = dept.DEPTNO;

Result Table:

ENAME	DEPTNO	DEPTNO	DNAME	
A	20	20	D2	} matching Records
B	30	30	D3	
C	NULL	NULL	NULL	} unmatched Records of left table
D	NULL	NULL	NULL	
NULL	NULL	10	D1	} unmatched Records of right table
NULL	NULL	40	D4	

④ NATURAL JOIN :-

It has two behaviours

- ① It behaves as Inner Joins when there is relation between the tables
- ② It behaves as CARTESIAN JOIN when there is no relation between the tables.

Syntax:-

Only ANSI Syntax:-

```
SELECT COL-NAME
FROM TABLE-NAME1 NATURAL JOIN TABLE-NAME2;
```


Q: WAQTD ename and reporting mgr name.
→

Self JOINS

- Joining a table itself, or joining some two tables
It is known as self joins

- when or why do we use self join
- whenever to be data to be selected present in some table but in different records then we use the concept of self joins

Syntax:

ANSI

```
SELECT column_name  
FROM Table_name1 JOIN Table_name2  
ON <JOIN_CONDITION>
```

ORACLE

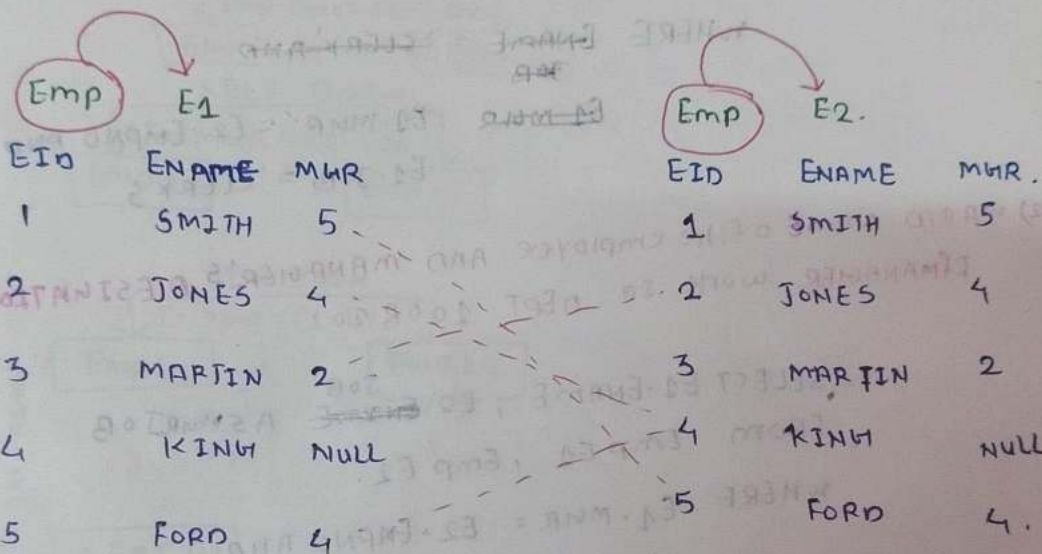
```
SELECT column_name  
FROM Table_name1, Table_name2  
WHERE <JOIN_CONDITION>
```

Q: WAQTD Ename & mgr reporting mgr name

```
③ SELECT E1.ENAME, E2.ENAME AS MGR_NAME  
① FROM Emp E1, Emp E2.  
② WHERE E1.MGR = E2.ENO;
```

JOIN-condition

$5=1 (F)$ $4=1 (F)$ $2=1 (F)$ $NULL=1 (F)$ $4=1 (F)$
 $5=2 (F)$ $4=2 (F)$ $2=2 (T)$ $NULL=2 (F)$ $4=2 (F)$
 $5=3 (F)$ $4=3 (F)$ $2=3 (F)$ $NULL=3 (F)$ $4=3 (F)$
 $5=4 (F)$ $4=4 (T)$ $2=4 (F)$ $NULL=4 (F)$ $4=4 (T)$
 $5=5 (T)$ $4=5 (F)$ $2=5 (F)$ $NULL=5 (F)$ $4=5 (F)$



GROUP OF SELECT CLAUSE:

ENAME MGR-NAME
 SMITH FORD
 JONES KING
 MARTIN JONES
 FORD KING

Q1- WAGTD NAME of the employees AND HIS MANAGER'S NAME
If employees is working AS CLERK.

```
SELECT E1.ENAME, E2.ENAME AS MGR-NAME  
FROM Emp E1, Emp E2  
WHERE E1.MGR = E2.EMPNO
```

E1.MGR = E2.EMPNO AND
E1.JOB = 'CLERK';

Q2) WAGTD NAME of the employee AND MANAGER'S DESIGNATION
If MANAGER work IN DEPT 10 OR 20.

```
SELECT E1.ENAME, E2.JOB AS MGRJOB  
FROM Emp E1, Emp E2.
```

WHERE E1.MGR = E2.EMPNO AND
E2.DEPTNO IN (10, 20);

Q3- WAGTD NAME of the Emp AND MANAGERS SALARY If employee
AND MANAGER BOTH EARN MORE THAN 2300

```
SELECT E1.ENAME, E2.SAL AS MGR-SAL  
FROM Emp E1, Emp E2
```

WHERE E1.MGR = E2.EMPNO AND

E1.SAL > 2300 AND

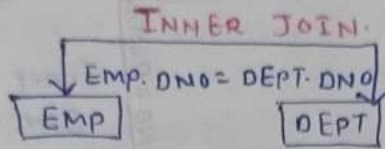
E2.SAL > 2300;

TO JOIN MULTIPLE TABLES

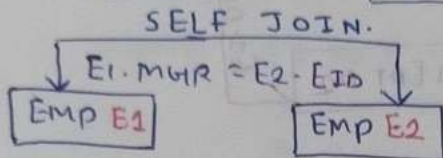
* To join n number of table we need to write n-1 join conditions

- we can join 256 tables

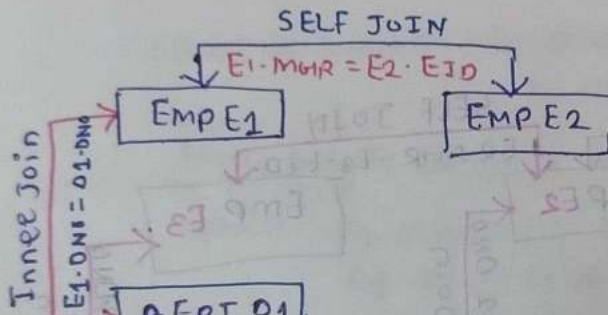
①



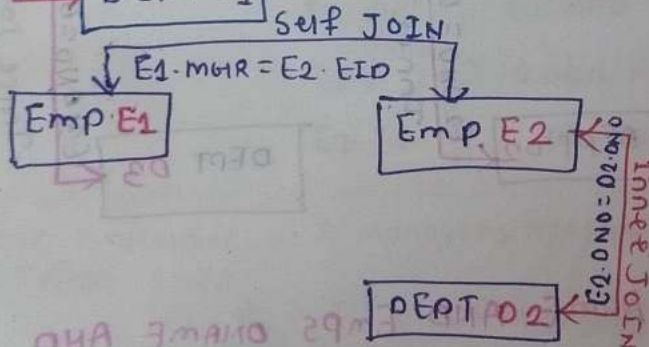
②



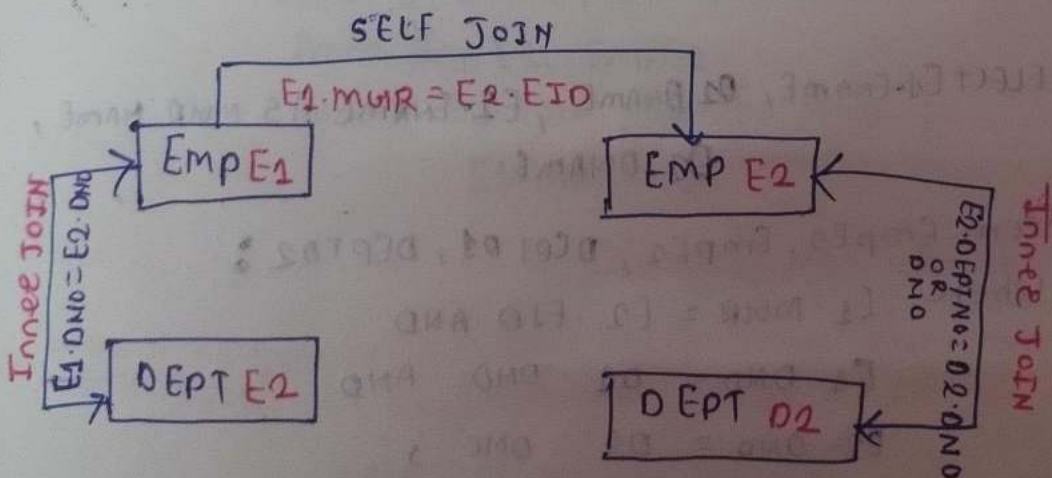
③



④

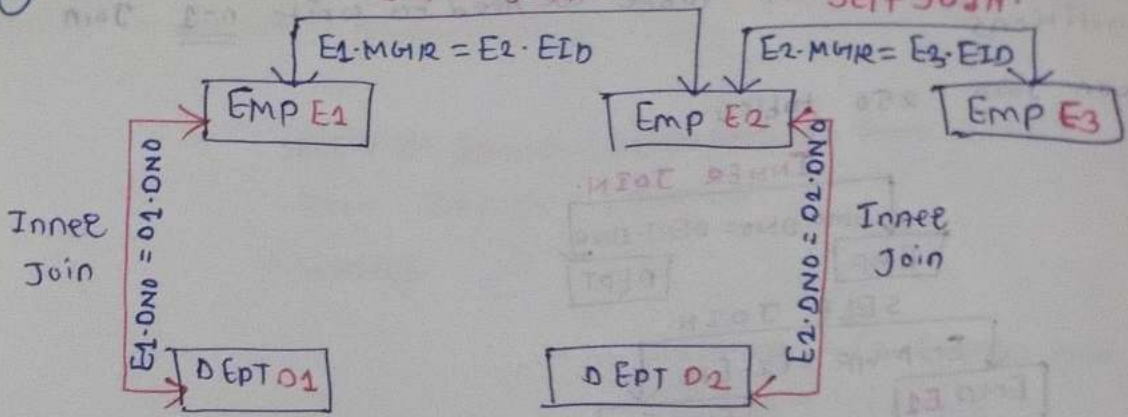


⑤

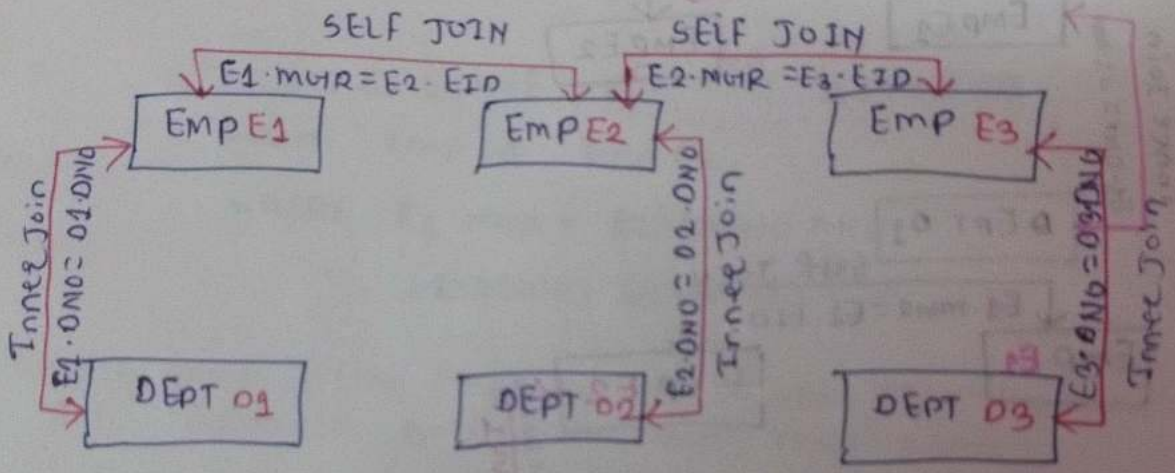


6

TO JOIN MULTIPLE TABLES SELF JOIN



7



Q:- WAOT ENOME, MANAGER NAME AND EMPS ONAME AND MANAGERS DNAME.

```

SELECT E1.ENAME, D2.DNAME, E1.ENAME AS MGR_NAME,
       D2.DNAME
FROM EMP E1, EMP E2, DEPT D1, DEPT D2
WHERE E1.MANAGER = E2.EMPID AND
       E1.DEPTNO = D1.DEPTNO AND
       E2.DEPTNO = D2.DEPTNO ;
    
```


Q: WAO TO EMP ENAME, MANAGER'S NAME AND THEIR ONAMES IF EMP EARN MORE THAN 2000 AND MANAGER WORKING IN DEPT 20.

~~SELECT E1.ENAME, E2.ENAME AS MGR_ENAME,
D1.DNAME, D2.DNAME
FROM EMP E1, EMP E2, DEPT D1, DEPT D2
WHERE E1.MGR = E2.EMPNO AND
D1.DNAME
D1.DEPTNO = D2.DEPTNO AND~~

SELECT E1.ENAME, E2.ENAME AS MGR_ENAME,
D1.DNAME, D2.DNAME
FROM EMP E1, EMP E2, DEPT D1, DEPT D2.

WHERE E1.MGR = E2.EMPNO AND
E1.DNO = D1.DNO AND
E2.DNO = D2.DNO AND
E1.SAL > 2000 AND
E2.DEPTNO = 20;

Q: WAO TO EMP NAME AND MANAGER'S HIREDATE IF EMPLOYEES WAS HIRED BEFORE 1982

SELECT E1.ENAME, E2.HIREDATE AS MGR_HIREDATE
FROM EMP E1, EMP E2
WHERE E1.MGR = E2.EMPNO AND
E1.HIREDATE <

'01-JAN-1982';
OR
<= '31-DEC-1981';

Q: WAO TO EMP NAME AND MANAGER'S COMM IF EMPLOYEE WORKS AS SALESMAN AND MANAGER WORKS IN DEPT 30.

SELECT E1.ENAME, E2.COMM AS MGR_COMM
FROM EMP E1, EMP E2
WHERE E1.MGR = E2.EMPNO AND
E1.JOB = 'SALESMAN' AND
E2.DEPTNO = 30;

Q:- WAO TO EMP ENAME AND MANAGER NAME AND THEIR SALARIES
 IF EMPLOYEES SOME MANAGER THAN THEIR MANAGER NAME, EMP ENAME
 AND SALARY

```
SELECT E1.ENAME, E2.ENAME AS MGR_NAME,
       E1.SAL, E2.SAL AS MGR_SAL
FROM EMP E1, EMP E2
WHERE E1.MGR = E2.EMPNO AND
       E1.SAL > E2.SAL;
```

Q:- WAO TO EMP NAME AND HIREDATE, MANAGER NAME AND HIREDATE
 IF MANAGER WAS HIRED BEFORE EMPLOYEES

```
SELECT E1.ename, E1.HIREDATE, E2.ENAME AS MGR_NAME,
       E2.HIREDATE AS MGR_HIREDATE
FROM EMP E1, EMP E2
WHERE E1.MGR = E2.EMPNO AND
       E2.HIREDATE < E1.HIREDATE;
```

Q:- WAO TO emp name and managers name if both are working AS TO
 SAME JOB

```
SELECT E1.ename, E2.ename AS MGR_name
FROM EMP E1, EMP E2
WHERE E1.MGR = E2.EMPNO AND
       E1.JOB = E2.JOB;
```

Q:- WAO TO EMP ENAME AND MANAGER name if manager is working AS
 Actual manager

```
SELECT E1.ename, E2.ename AS MGR_ename
FROM EMP E1, EMP E2
WHERE E1.MGR = E2.EMPNO AND
       E2.JOB = 'MANAGER';
```


SALARIES
NAME, OTAW
1970
NAME,
AL.

HIREDATE
AS MGR_NAME
HIREDATE.

AS TO
OTAW
1970

king AS
OTAW
AME

WANT EMP NAME AND MANAGER NAME ALONG WITH THEIR ANNUAL SALARY
WANT EMPLOYEE'S NAME AND MANAGER'S SALARY
WANT EMPLOYEE'S SALARY AND MANAGER'S NAME

```
SELECT E1.ename, E2.ename AS MGR_ename,
       E1.sal*12, E2.sal*12 AS MGR_Annualsal
FROM emp E1, emp E2
WHERE E1.mgr = E2.empno AND
      E1.deptno IN (10,20) AND
      E2.sal > E1.sal;
```

WANT employee's name AND MANAGER'S designation for all the employees

```
SELECT E1.ename, E2.job AS MGR-JOB
FROM emp E1, emp E2
WHERE E1.mgr = E2.empno;
      E2.job = 'EMP';
```

WANT employee's name and manager's salary for all the employee
if manager's salary ends with 50

```
SELECT E1.ename, E2.sal AS MGR_SAL
FROM emp E1, emp E2
WHERE E1.mgr = E2.empno AND
      E2.sal LIKE '%50';
```

WANT ename, manager, name and their ename if emp coming
more than smith and manager earning more than ALLEN

```
SELECT E1.ename, E2.ename AS MGR_NAME, D1.ename, D2.ename
FROM emp E1, emp E2, dept D1, dept D2
WHERE E1.mgr = E2.empno AND
      E1.dno = D1.dno AND
      E2.dno = D2.dno AND
      E1.sal > (SELECT sal
                FROM emp
                WHERE ename = 'SMITH')
      AND E2.sal > (SELECT sal
                    FROM emp
                    WHERE ename = 'ALLEN');
```

(OR)
E1.sal > (SELECT sal
FROM emp
WHERE ename = 'SMITH') AND
E2.sal > (SELECT sal
FROM emp
WHERE ename = 'ALLEN');

From Emp E1
EMP
subquery

DNAME
AS
MGR_DNAME

E1.sal > (SELECT sal
FROM emp
WHERE ename = 'SMITH')
AND E2.sal > (SELECT sal
FROM emp
WHERE ename = 'ALLEN');

Q:- Write a query to find the name and salary of the employee working in DEPT 10 OR 30 AND MANAGER earning more than 1000 AND Emp working IN LOC NEWYORK OR CHICAGO.

OR
 E2.SAL > (SELECT SAL EMP E1
 FROM EMP
 WHERE ENAME = 'Ford')
 FROM EMP E1, EMP E2, DEPT D1, DEPT D2
 WHERE E1.MGR = E2.EMPNO AND
 E1.DNO = D1.DNO AND
 E2.DNO = D2.DNO AND
 E1.DNO IN (10, 30) AND
 E1.EMPNO IN ('NEWYORK', 'CHICAGO');

Q:- Write a query to find the name and salary of the employee working in DEPT 10 OR 30 AND MANAGER earning more than 1000 AND Emp working IN LOC NEWYORK OR CHICAGO.

SELECT E1.ename, E2.ename MGR-NAME, E3.ename
 FROM EMP E1, EMP E2, EMP E3
 WHERE E1.MGR = E2.EMPNO AND
 E2.MGR = E3.EMPNO

Q:- Write a query to find the name and salary of the employee working in DEPT 10 OR 30 AND MANAGER earning more than 1000 AND Emp working IN LOC NEWYORK OR CHICAGO.

SELECT E1.ename, E2.ename MGR-NAME, E3.ename AS MGR-NAME
 D1.DNAME, D2.DNAME, D3.DNAME
 FROM EMP E1, EMP E2, EMP E3, DEPT D1, DEPT D2, DEPT D3
 WHERE E1.MGR = E2.EMPNO AND
 E2.MGR = E3.EMPNO AND
 E1.DNO = D1.DNO AND
 E2.DNO = D2.DNO AND
 E3.DNO = D3.DNO AND

E1.SAL > 1000 AND

E2.SAL > CSELECT SAL
From E1.Emp

where E1.ename = 'ALLEN') AND

OR E2.SAL > CSELECT SAL
From Emp
where Ename = 'ALLEN'

D3.LOC IN ('New York', 'CHICAGO')

Q:- want to name manager's name AND managers manager name along with their loc IF the emp HIRED before MARTIN AND MANAGER working in Accounting OR SALES Dept AND MANAGER earning managers earning SAL more than Smith's

SELECT E1.ename, E2.ename AS MGR_name, E3.ename AS
MGR_name

D1.LOC, D2.LOC, D3.LOC

From Emp E1, Emp E2, Emp E3, DEPTNO D1, DEPTNO D2, DEPTNO D3

where E1.MGR = E2.ENO AND

E2.MGR = E3.ENO AND

E1.DNO = D1.DNO AND

E2.DNO = D2.DNO AND

E3.DNO = D3.DNO AND

E1.HIREDATE < CSELECT E1.HIREDATE

From E1.Emp

where E1.ename = 'MARTIN') AND

D2.DNAME IN ('ACCOUNTING', 'SALES') AND

E3.SAL > CSELECT E1.SAL

From

where Ename = 'SMITH';

OR

E3.SAL > CSELECT SAL
From emp

where Ename = 'SMITH';

OR E3.ename

29/3/25

Q: WHAT IS SALARY IN ASCENDING ORDER.

SELECT SAL

FROM EMP

ORDER BY SAL ASC;

→ order by clause is compulsory but it is possible to use it

* ORDER BY CLAUSE *

- It is used to sort the record in Ascending or descending order.
- It executes after execution of select clause.
- ORDER BY CLAUSE must be return as a last clause in the statement.
- For order by clause, we can pass column name or expression as an argument.
- For order by clause, we can pass Alias name AS an argument.
- By default order by clause, sort the Record in Ascending order.
- we can pass multiple Argument to order by clause but it will sort the Records in combination.

Syntax:

SELECT Group-Function / Group-by-Exp / col-name

FROM TABLE-NAME

[WHERE <filter-condition>]

[GROUP BY col-name / Expression]

[HAVING <Group-filter-condition>]

ORDER BY col-name / EXP [ASC] / DESC;

- 1- FROM
- 2- WHERE
- 3- GROUP BY
- 4- HAVING
- 5- SELECT
- 6- ORDER BY

Q: WHAT IS

② SELECT

① FROM

③ WHERE

④ GROUP BY

⑤ HAVING

⑥ ORDER BY

⑦ SELECT

⑧ FROM

⑨ WHERE

⑩ GROUP BY

⑪ HAVING

⑫ ORDER BY

⑬ SELECT

⑭ FROM

⑮ WHERE

⑯ GROUP BY

⑰ HAVING

⑱ ORDER BY

⑲ SELECT

⑳ FROM

㉑ WHERE

㉒ GROUP BY

㉓ HAVING

㉔ ORDER BY

㉕ SELECT

㉖ FROM

㉗ WHERE

㉘ GROUP BY

㉙ HAVING

㉚ ORDER BY

㉛ SELECT

㉜ FROM

㉝ WHERE

㉞ GROUP BY

㉟ HAVING

㊱ ORDER BY

㊲ SELECT

ORDER OF Execution

- 1- FROM
- 2- WHERE → Row-by-Row
- 3- GROUP BY → Row-by-Row
- 4- HAVING → Group-by-group
- 5- SELECT → Group-by-group
- 6- ORDER → Group-by-group

Q:- WRTD SAL IN ASCENDING ORDER.

② SELECT SAL

① FROM Emp

③ ORDER BY SAL [Asc];

Database

o/p of
From clause.

EID	ENAME	SAL	DNO
1	SIDDHI	3000	20
2	SIDHU	1500	10
3	HARSHADA	2000	20
4	HARSHAD	2000	20
5	CHAN CHAL	3000	10

o/p of SELECT clause

SAL
3000
1500
2000
2000
3000

o/p of ORDER BY

SAL
1500
2000
2000
3000
3000

Q:- WRTD Enome In Asc. ORDER.

SELECT Enome
From emp

ORDER BY ENAME Asc;

Q:- WANTO SAL IN DESC ORDER.

```
SELECT SAL
FROM emp
ORDER BY SAL DESC;
```

Q:- WANTO ENAME IN DESC ORDER.

```
SELECT ENAME
FROM emp
ORDER BY ENAME DESC;
```

Q:- WANTO ENAME AND DEPTNO IN ASC ORDER.

```
SELECT ENAME, DEPTNO
FROM emp
ORDER BY ENAME, DEPTNO ASC;
```

Q:- WANTO ANNUAL SAL IN DESC ORDER

```
SELECT SAL*12 ANNUAL-SAL
FROM emp
ORDER BY SAL*12 DESC;
```

EMPNO	ENAME	SAL	ANNUAL-SAL
7782	CLERK	1200	14400
7781	CLERK	1300	15600
7783	CLERK	1900	22800
7784	CLERK	2400	28800
7785	CLERK	3000	36000

SAL
1200
1300
1900
2400
3000

SAL
1200
1300
1900
2400
3000

It is used to
There are

- ① CRE
- ② RE
- ③ A
- ④ T
- ⑤

*CREATE :-

DOL

(Data Definition Language)

It is used to construct, modify, and Remove an object.

There are five statements

- ① CREATE
- ② RENAME
- ③ ALTER
- ④ TRUNCATE
- ⑤ DROP

Note:

- This Statement Are Auto comment Statement

→ Auto comment

CREATE :- This statement is used to construct an object (Table)

Syntax:

CREATE TABLE TABLE_NAME

(
 col-NAME1. DATATYPE constraints,
 col-NAME2. DATATYPE constraints,
 col-NAME3. DATATYPE constraints,
 :
 col-NAME4. DATATYPE constraints,
);

TABLE_NAME : IPL-TEAM

NO. of column : 05

DATATYPE	NUMBER(5)	VARCHAR	VARCHAR	NUMBER(10)	VARCHAR
COL-NAME	TID	T-NAME	O-NAME	MOB-NO	EMAIL
UNIQUE		UNIQUE		UNIQUE	UNIQUE
NOTNULL		NOT-NULL	NOT-NULL	NOT-NUL	NOT-NULL
CHECK	CHECK (LENGTH(TID)=5)			CHECK (LENGTH(MOB)=10)	
Primary key	PK				
Foreign key					



Constraints.


```

CREATE TABLE IPL-TEAM
(
  TID NUMBER(5) PK CHECK (LENGTH(TID)=5)
  PRIMARY KEY
  TNAME VARCHAR(8) UNIQUE NOT NULL
  QNAME VARCHAR(15) NOT NULL
  MOB-NO NUMBER(10) UNIQUE NOT NULL CHECK (LENGTH(MOB-NO)=10)
  EMAIL VARCHAR(15) UNIQUE NOT NULL);

```

* To create Table with Foreign Key.

Syntax:

```

CREATE TABLE TABLE_NAME
(
  col-name1 Datatype constraints,
  col-name2 Datatype constraints,
  col-name3 Datatype constraints,
  ...
  col-nameN Datatype constraints,
  constraints ref-name Foreign Key (col-name)
  References Table-name (col-name)
);

```

blueprint

TABLE_NAME : PLAYERS
NO. of column : 06

Datatype	Numbers(7)	VARCHAR(15)	Numbers(2)	Numbers(10)	VARCHAR(15)	Number(5)
col-name	PID	Pname	AGE	MOB-NO	EMAIL	NO-MATCHES
UNIQUE				UNIQUE	UNIQUE	
NOT NULL		NOT NULL	NOT NULL	NOT NULL	NOT NULL	NOT NULL
CHECK				CHECK (length(mobno)=10)		
PK	PK					

FK
↓
constraints

Query:
CREATE TABLE
(
PID NUMBER
PNAME VARCHAR
AGE NUMBER
MOB-NO
EMAIL
NO-MATCHES
TID NUMBER
constraints
References
);

* To copy A

Ex: 23 JAG

* TO CREATE
- It is out
- It will
- Any ch

Query 2

CREATE TABLE PLAYERS

(
PID NUMBER (7) PRIMARY KEY,
PNAME VARCHAR (15) NOTNULL,
AGE NUMBER (2) NOTNULL,
MOB_NO NUMBER (10) UNIQUE NOTNULL CHECK (LENGTH(MOB_NO)=10),
EMAIL VARCHAR (15) UNIQUE NOTNULL,
NO-MATCHES NUMBER (3) NOTNULL,
TID NUMBER (5),
constraint TID-FK Foreignkey (TID)
REFERENCES IPL-TEAM (TID)
);

* TO COPY A TABLE :

Syntax :

CREATE TABLE TABLE_NAME
AS
SELECT SQL-STATEMENT;

Ex: 2.2.2.2

CREATE TABLE SALESMAN

AS

SELECT *

From emp

where JOB = 'SALESMAN';

* TO CREATE VIEW :

- It is virtual table which is created on the Result of the query.
- It will not occupied storage.
- Any changes Done on the original table it will Impact on view.

Syntax :

CREATE VIEW VIEW_NAME
AS
SQL-Statement;

CREATE VIEW MANAGER

AS

SELECT *

From emp

where JOB = 'MANAGER';

number number
(5)

MATCHES TID

NOTNULL

FK

② RENAME :

It is used to change the existing object name.

Syntax :

RENAME old-obj-NAME TO NEW-obj-NAME;

③

ex:-

RENAME STUD TO SUPERMAN;

③ ALTER :

④ TRUNCATE :

It is used to remove all the records from the tables permanently.

Syntax :

TRUNCATE TABLE TABLE-NAME;

ex:- TRUNCATE TABLE SALESMAN;

- DESC SALESMAN
- Select *
From salesman.

⑤ DROP : It is used to remove an object (TABLES)

Syntax :

DROP TABLE TABLE-NAME;

FLASH

ex:-

DROP TABLE SUPERMAN;

* FLASHBACK (TO RECOVER THE TABLE)

Syntax :

FLASHBACK TABLE TABLE-NAME
TO BEFORE DROP;

आपण जर table
block केला तर
परत Flashback
नी परत सेतो.

ex:-

FLASHBACK TABLE SUPERMAN
TO BEFORE DROP;

* PURGE (TO delete From Recyclebin)

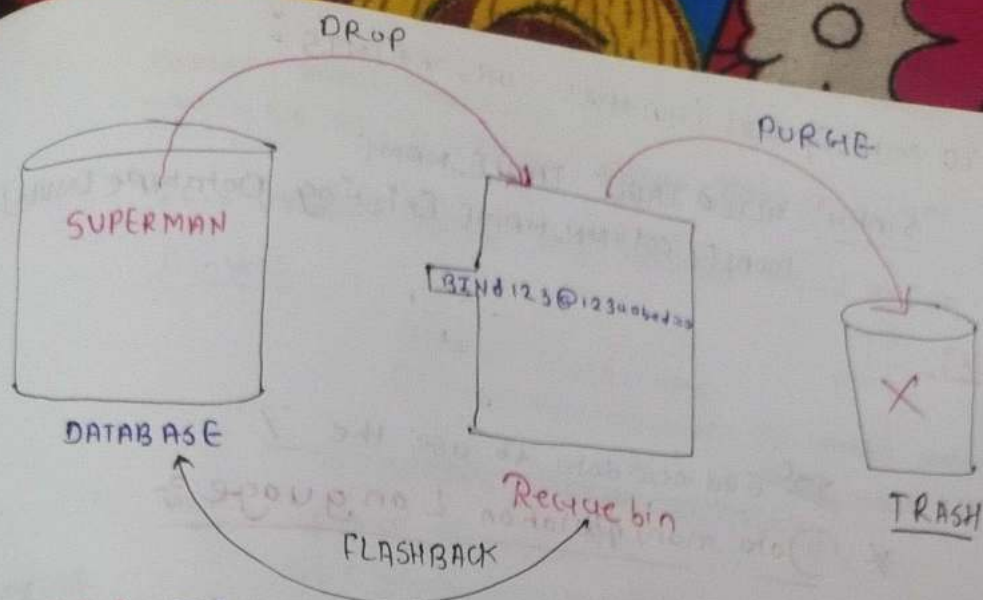
Syntax :

PURGE TABLE TABLE-NAME;

ex:-

PURGE TABLE SUPERMAN;

जगि आपण पुन्हाही गोर
delete केली तर ती
Recyclebin मध्ये जाते
त्यानून पुण delete
करायची तर
PURGE
use करायचा.



③ ALTER :- This statement is used to modify the existing object

① To ADD A COLUMN :-

Syntax:- ALTER TABLE TABLE_NAME
ADD column_name DATATYPE constraint_type;

ex:-

② TO DROP A COLUMN :-

Syntax:- ALTER TABLE TABLE_NAME
DROP COLUMN COLUMN_NAME;

ex:-

③ TO RENAME A COLUMN :-

Syntax :- ALTER TABLE TABLE_NAME
RENAME COLUMN COLUMN_NAME TO NEW_COLUMN_NAME;

ex:-

④ TO MODIFY THE DATATYPE :-

Syntax:- ALTER TABLE TABLE_NAME
MODIFY COLUMN_NAME NEW_DATATYPE;

ex:-

⑤ TO MODIFY NULL / NOT NULL CONSTRAINTS :

Syntax: ALTER TABLE TABLE-NAME
MODIFY COLUMN-NAME Existing-Datatype (NULL) / NOT NULL

ex:

Imp add new data to use the '/'

* Data Manipulation Language :

- This statement is used to manipulate the object by performing insertion, update, and initialization.
- There are 3 statements in DML.

① INSERT

② UPDATE

③ DELETE

① INSERT :

This statement is used to INSERT the Record's or add the Records into the table.

Syntax:

INSERT INTO TABLENAME
VALUES (V1, V2, V3, ... Vn);

ex:

INSERT INTO IPL-TEAM

VALUES (12435,

MI,

AMBANI,

12345678)

AMBANI@gmail.com);

Syntax:

INSERT INTO TABLE-NAME (

TABLE-NAME (COL1, COL2, ... COLn)

VALUES (V1, V2, V3, ... Vn);

ex:

INSERT INTO IPL-TEAM (TID, TNAME, O-NAME, MOB.NO, EMAIL)
VALUES (12342, 'ESK', 'SHRINIVAS', 8123456789, SINDU@gmail.com);

Syntax:

INSERT INTO TABLE-NAME

VALUES (fcol1, fcol2, fcol3, ... fcoln);

ex: INSERT INTO IPL-TEAM
VALUES (TID, TNAME, O-NAME, MOG-NO, EMAIL)

UPDATE:

It is used to update the existing record or data.

Syntax:

UPDATE TABLE-NAME
SET col-name1=value, col-name2=value, --
col-N=VALUE

WHERE < filter-condition >;

ex:

UPDATE IPL-TEAM
SET O-NAME = 'RUSHI'
WHERE TID = 12342;

Don't
use
AND operator
IN OR

UPDATE IPL-TEAM
SET O-NAME = 'ADITYA'
WHERE TID IN (12341, 12344)

③ DELETE:

This statement is used to delete the Particular Records.

Syntax:

DELETE FROM TABLE-NAME
WHERE < filter-condition >.

ex:

DELETE FROM IPL-TEAM
WHERE TID = 12344;

* TCL *

* Transactional Control Language *

* There are three statements in TCL:

- ① commit
- ② SAVEPOINT
- ③ ROLLBACK:

① commit:

- It is used to save the transaction.

Syntax: commit;

② SAVEPOINT:

- It is used to mark the transaction.
- It can be used along with Rollback.

Syntax:

SAVEPOINT SAVEPOINT-NAME;

③ Rollback:

- It is used to get back the latest saved transaction.

Syntax:

ROLLBACK;

Syntax:

Rollback To Savepoint-NAME;

Example of savepoint

SAVEPOINT P1;

Example of Rollback:

ROLLBACK TO P1;

Data Control Language

It is used to control the flow of data.

There are 2 statements in Data Control Language

- ① GRANT
- ② REVOKE

username . tablename

① GRANT :- It is used to give permission to a user

Syntax :-

GRANT sql_statement ON Table-NAME
TO USER-NAME;

ex :- GRANT SELECT ON EMP
TO HR;

② REVOKE :- It is used to take back the given permission from user.

Syntax :-

REVOKE sql_statement ON Table-NAME
FROM User-NAME;

ex :- REVOKE SELECT ON EMP
FROM HR;

To connect other user / To jump from one user to another.

connect /conn
USERNAME: SCOTT | HR

PASSWORD: TIGER → smart

Basic command-

command clear

- 1) To log in or to connect two user

username:	
Password:	
Host string:	
OK	Cancel

edit user
user name

set pages 150 lines 150
select * from emp;

- 2) To see current user

Syntax

show user;

Example

show user;

Select x

From TAB;

- 3) To switch between user or account

4) Syntax

1) connect username;
enter password: pass;

ex

connected system;
enter pass: tiger

a) conn username;
enter pass word: pass

conn scott;
enter pass: Tiger

- 4) To set page size sql console

Syntax

set page page_size;

ex

set page 1000;

- 5) To set line size in sql console

Syntax

set lines line_size;

set line 100;

- 6) To set page size and line size; set pages 1000 of sql console
Together

Syntax

Example-

set page page_size lines line_size; set page 1000 lines 1000

- 7) To see the list of user or account's present in database.

Syntax

SELECT * from All user

Example

SELECT * from
All-user

- 8) To see list of tables present in an account.

Syntax.

SELECT * from TABS;

Example

SELECT * from TABS;

To see the structure of the table

Syntax

- 1) Describe table_name
- 2) desc table_name

Example

- 1) Describe dept
- 2) desc emp

To see the details of all the records present in the tables

Syntax

SELECT * FROM table_name;

Example

SELECT * FROM emp;

To edit the last executed query

Syntax

- 1) Edit
- 2) Ed

Example

- 1) Edit
- 2) Ed

- 1) To call the editor follow the above syntax.
- 2) The editor will by default reflect last executed query.
- 3) The editor allow you to make changes in any line of the query.
- 4) After making changes, save those changes by using Control + S or click on save option from file menu. and close the editor.
- 5) The edited query will now reflect on sql console.
- 6) To execute this query use Forward (/) slash button.

Syntax /

To clear SQL console screen

Syntax

- 1) clear screen
- 2) clscr

Example

Q: What is the name of the employee who are earning more than 10000

Select empname

from emp

where sal > 10000

Q: What is the name of employees who are earning more than 10000

Select empname

from emp

where sal > 10000

Select sal

from emp

where empname = 'John'

Employee Table -

Yuvraj Vinayak Pol

```
SQL> set lines 100 pages 100
SQL> select * from emp;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

14 rows selected.

Department Table -

Yuvraj Vinayak Pol

```
SQL> select * from dept;
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON